

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Facultad de Ingeniería
Escuela de Ciencias y Sistemas

MANUAL TÉCNICO

API REST, vistas y triggers
SQL Server 2022 en Docker

Comercial La Estrella · Fase 2

Estudiante: Alex Ricardo Castañeda Rodríguez
Carné: 202300476
Curso: Bases de Datos 1
Período: Segundo semestre 2026
Fecha: 16 de octubre de 2026

Guatemala, 2026

Índice

1. Propósito, alcance y entregables **4**
1.1. Correspondencia con el enunciado . . . 4
1.2. Cómo utilizar el manual . . . 4

2. Arquitectura de la solución **5**
2.1. Responsabilidad de cada capa . . . 5
2.2. Recorrido de una petición de venta . . . 5

3. Organización del proyecto **6**

4. Requisitos y preparación del equipo **6**
4.1. Herramientas . . . 6
4.2. Obtener el código . . . 6

5. Configuración de variables y credenciales **7**
5.1. Creación inicial . . . 7
5.2. Reglas de configuración . . . 7

6. Arranque de SQL Server con Docker **7**
6.1. Crear los servicios . . . 7
6.2. Qué hace la inicialización . . . 8
6.3. Persistencia y detención . . . 8

7. Compilación y ejecución de la API **8**
7.1. Arranque diario con la carga ya realizada . . . 9

8. Migración del modelo y del histórico **9**
8.1. Cambios entre Oracle y SQL Server . . . 9
8.2. Conteos de referencia . . . 9
8.3. Primera importación: solo sobre una instalación vacía . . . 10
8.4. Transacción, seguridad y conciliación . . . 10

9. Contrato HTTP, validación y errores **11**
9.1. Listados y paginación . . . 11
9.2. Tipos y restricciones de entrada . . . 11
9.3. Códigos HTTP y lectura de fallos . . . 12

10. TypeORM y acceso a datos **12**

11. Inventario: trigger Insert_Detalle **12**
11.1. Qué es un trigger . . . 12

11.2. Cálculo del ajuste . . . 13
11.3. Actualización atómica y rollback . . . 13
11.4. Demostración mínima . . . 13

12. Pagos: trigger Insert_Pago y estados **13**
12.1. Ejemplo de venta de Q50 . . . 13

13. Protecciones complementarias del negocio **14**
13.1. Validar_Venta . . . 14
13.2. Proteger_Catalogo y Proteger_Estados . . . 14
13.3. Resumen de dependencias . . . 14

14. Vistas y reportes SQL **14**
14.1. Vista_Ventas_Ubicacion . . . 14
14.2. Vista_Top_Productos . . . 14
14.3. Vista_Saldos_Pendientes y vista auxiliar . . . 15
14.4. Por qué se agregan detalles y pagos por separado . . . 15

15. Postman: importación, variables y ejecución **15**
15.1. Importar los archivos correctos . . . 15
15.2. Qué significan las llaves dobles . . . 15
15.3. Cómo repetir mañana la demostración . . . 16
15.4. Ejemplo manual de CRUD . . . 16
15.5. Colección de importación . . . 16

16. Lotes, atomicidad y concurrencia **16**

17. Pruebas y resultados registrados **16**
17.1. Comandos reproducibles . . . 16
17.2. Escenarios de integración . . . 17

18. Evidencias de inventario y pagos **17**
18.1. Inventario: rechazo, conservación y descuento . . . 17
18.2. Abonos y cambio automático de estado . . . 19
18.3. Comprobación de persistencia . . . 20

19. Inspección estructural con DBeaver **20**

20. Guion detallado para la demostración **20**
20.1. Preparación de una grabación . . . 21

21. Solución de problemas **21**

22. Seguridad, límites y entrega reproducible **22**
22.1. Documentación editable . . . 22

23. Preguntas de estudio y respuestas **23**

A. Referencia detallada de las 19 entidades **24**

A.1. pais . . . 24
A.2. departamento . . . 25
A.3. municipio . . . 26
A.4. tipo_tienda . . . 26
A.5. tipo_identificacion . . . 27
A.6. cargo . . . 27
A.7. categoria . . . 28
A.8. marca . . . 28
A.9. estado_venta . . . 29
A.10. metodo_pago . . . 29
A.11. persona . . . 30
A.12. tienda . . . 31
A.13. empleado . . . 32
A.14. cliente . . . 33
A.15. producto . . . 34
A.16. catalogo_producto . . . 35
A.17. venta . . . 36
A.18. detalle_venta . . . 37
A.19. pago . . . 38

B. Matriz de todas las peticiones de Postman **39**

B.1. 00 Preparacion . . . 39
B.2. 01 CRUD por entidad . . . 39
B.3. 02 Limpieza auxiliares . . . 44
B.4. 03 Triggers . . . 45
B.5. 04 Vistas . . . 45

C. Implementación SQL de las vistas y triggers **46**

C.1. Vista_Ventas_Ubicacion . . . 46
C.2. Vista_Top_Productos . . . 46
C.3. Vista_Consulta_Ventas . . . 46
C.4. Vista_Saldos_Pendientes . . . 47
C.5. Insert_Detalle . . . 47
C.6. Insert_Pago . . . 48
C.7. Validar_Venta . . . 49
C.8. Proteger_Catalogo . . . 49
C.9. Proteger_Estados . . . 50

D. Fuentes y trazabilidad **51**

1. Propósito, alcance y entregables

Este manual explica cómo instalar, ejecutar, verificar y defender la segunda fase de Comercial La Estrella. El sistema conserva el modelo de 19 tablas de la fase 1 y lo implementa en SQL Server 2022. La API REST utiliza Node.js, Express, TypeScript y TypeORM; el motor se ejecuta en Docker y sus reglas operativas se aplican mediante restricciones y triggers de T-SQL.

El cliente de demostración es Postman. El enunciado no exige una interfaz frontend. DBeaver se utiliza para inspección estructural de solo lectura; durante la calificación, las altas, modificaciones y eliminaciones se realizan mediante la API. El diseño está preparado para uso local, con la API y el puerto SQL publicados en la interfaz de loopback.

1.1. Correspondencia con el enunciado

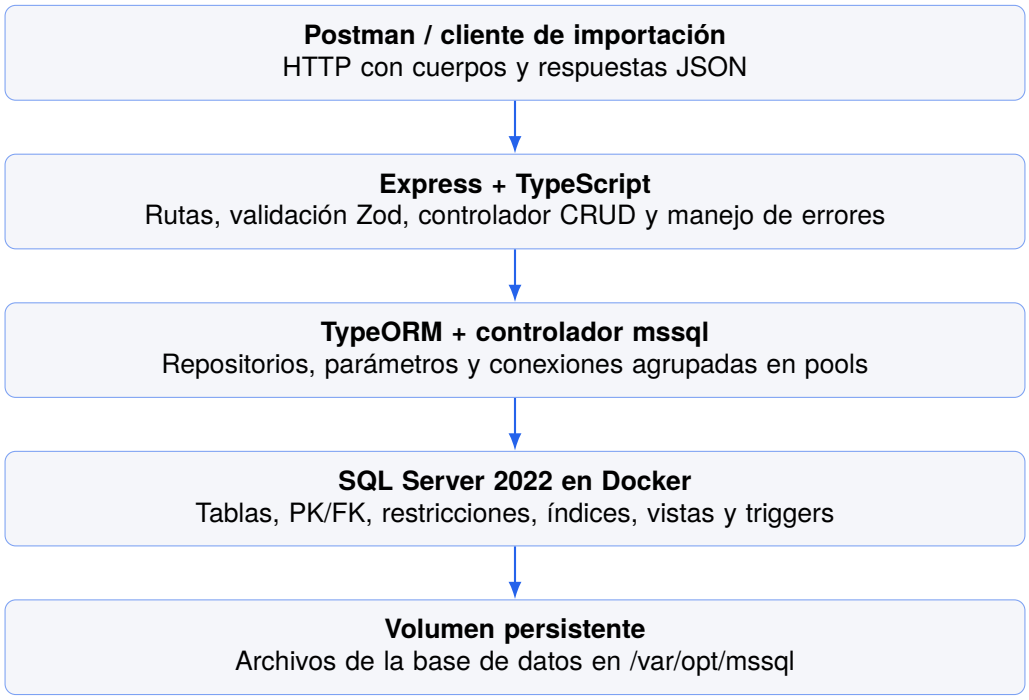
La sección 4 del enunciado solicita infraestructura, backend, colección de pruebas y documentación técnica. La documentación puede entregarse como README o PDF e incluye arranque del contenedor, ejecución del servidor y evidencias de los triggers. La hoja de calificación asigna 15 puntos a documentación.

Requisito	Implementación y ubicación
Infraestructura y DDL	`docker-compose.yml`, `init.sql` y `docker/init.sh`.
Backend completo	`src/`, `package.json`, `package-lock.json` y `tsconfig.json`.
CRUD de las 19 tablas	Módulos en `src/routes/`; controlador compartido en `src/controllers/crud.ts`.
Vistas y triggers	Definidos en `init.sql`; los reportes se consultan por HTTP.
Postman	Colección principal, entorno local y colección de importación en `postman/`.
Documentación y evidencias	Este manual, README, referencia de API y `docs/evidencias/`.
Repositorio	https://github.com/ricardious/SBD1_2S2026.

1.2. Cómo utilizar el manual

Para ejecutar en una máquina nueva, seguir las secciones de requisitos, configuración, Docker, API e importación. Para volver a demostrar una instalación existente, utilizar el apartado de arranque diario y la guía de Postman. La referencia de entidades detalla campos y ejemplos; el anexo de peticiones relaciona cada operación con su código HTTP esperado. Los listados de vistas y triggers reproducen el código de la implementación.

2. Arquitectura de la solución



2.1. Responsabilidad de cada capa

- **Postman:** construye peticiones, sustituye variables y evalúa aserciones de respuesta. No se conecta directamente al motor.
- **Express:** recibe HTTP, interpreta JSON y dirige la petición al módulo correspondiente. El cuerpo máximo configurado es 10 MB.
- **Zod:** comprueba tipos, rangos, fechas, claves y campos permitidos antes de escribir.
- **Controlador CRUD:** aplica el contrato común y usa repositorios; los casos especiales de venta y pago están explicitados.
- **TypeORM:** convierte objetos a operaciones SQL. Los metadatos se declaran con EntitySchema; no se activa sincronización automática.
- **SQL Server:** protege relaciones y reglas de negocio. El descuento de inventario y el estado de pago pertenecen al motor, no a un cálculo paralelo en Node.
- **Docker Compose:** coordina el motor y la inicialización, el volumen, la comprobación de salud y la publicación del puerto.

2.2. Recorrido de una petición de venta

Un POST a detalle de venta llega a Express; la ruta obtiene sus metadatos, Zod valida el cuerpo y el repositorio ejecuta INSERT. SQL Server evalúa restricciones y activa Insert_Detalle en la misma transacción. Si existe stock, descuenta y confirma; si falta stock, el trigger lanza un error y se revierte toda la sentencia. El middleware convierte el error de negocio en HTTP 400 con el mensaje Stock insuficiente. Tras una escritura correcta se consulta el registro y se devuelve HTTP 201.

3. Organización del proyecto

```
Proyecto 2/
  .env.example             Plantilla sin credenciales reales
  docker-compose.yml       SQL Server y servicio db-init
  init.sql                 Tablas, permisos, vistas y triggers
  docker/init.sh           Ejecucion automatica del DDL
  data/                    Historico JSON y manifiesto
  src/
    config/               Entorno y conexiones SQL
    entities/              Metadatos y EntitySchema
    controllers/           CRUD y lotes
    middleware/            Errores HTTP
    routes/                Entidades, importacion y reportes
    services/              Validacion e importacion
    app.ts                 Aplicacion Express
    server.ts              Inicio y cierre de conexiones
  postman/                 Colecciones y entorno
  tests/                   Pruebas unitarias y de integracion
  scripts/                 Configuracion, importacion y entrega
  docs/                    Enunciado, guias, evidencias y manual
```

La carpeta node_modules contiene dependencias instaladas y dist contiene JavaScript compilado. Ninguna se versiona. El archivo .env es local y privado. El histórico de datos sí forma parte de la entrega para reproducir la importación.

4. Requisitos y preparación del equipo

4.1. Herramientas

Se requiere Linux x86-64 con Docker Engine y el complemento Docker Compose, o Docker Desktop con contenedores Linux; Node.js 22 o superior; npm; Python 3 para las utilidades de preparación; y Postman. Python no interviene en las peticiones normales del backend. Para recompilar este documento se necesita XeLaTeX y las fuentes TeX Gyre Heros y TeX Gyre Cursor.

SQL Server tiene configurado un presupuesto interno de memoria de 2048 MB. No es un límite cgroup del contenedor ni una garantía sobre el consumo total. Se necesita margen adicional para el sistema operativo, la API, Postman, DBeaver y cualquier grabación. Los recursos disponibles dependen del resto de aplicaciones en ejecución.

```
docker --version
docker compose version
node --version
npm --version
python3 --version
```

Los puertos predeterminados son 1433 para SQL Server y 3000 para la API. Si están ocupados, ajustar .env y la variable baseUrl de Postman. Ejecutar los comandos de este manual desde la raíz de Proyecto 2, salvo que se indique lo contrario.

4.2. Obtener el código

```
git clone https://github.com/ricardious/SBD1_2S2026.git
cd "SBD1_2S2026/Proyecto 2"
```

Si ya se dispone de la carpeta local, abrir una terminal directamente en ella. El nombre contiene un espacio; por eso se entrecomilla al utilizar cd.

5. Configuración de variables y credenciales

5.1. Creación inicial

```
python3 scripts/setup-env.py
npm ci
```

El generador toma la plantilla .env.example, genera secretos aleatorios y crea .env con permisos privados. Si el archivo existe, no lo sobrescribe. npm ci instala exactamente el árbol del archivo package-lock.json. Las contraseñas de este manual no se incluyen: cada instalación usa sus propias credenciales.

Variable	Propósito y valor de referencia
MSSQL_SA_PASSWORD	Contraseña administrativa del motor, usada por Docker y db-init.
DB_HOST / DB_PORT	Conexión del backend: 127.0.0.1 y 1433 por defecto.
DB_NAME	ComercialLaEstrella
DB_USER / DB_PASSWORD	Login estrella_api y su contraseña; operaciones habituales.
IMPORT_DB_USER	estrella_import
IMPORT_DB_PASSWORD	Contraseña del login separado para la carga histórica.
IMPORT_KEY	Secreto de al menos 32 caracteres, enviado en X-Import-Key.
IMPORT_ENABLED	false normalmente; true solo durante la primera carga.
HOST / PORT	Dirección de escucha de la API: 127.0.0.1 y 3000.

5.2. Reglas de configuración

DB_PASSWORD e IMPORT_DB_PASSWORD deben tener de 16 a 128 caracteres de los conjuntos A-Z, a-z, 0-9, guion, guion bajo y signo de exclamación. init.sh valida ese conjunto antes de sustituir valores en los literales de sqlcmd. Las variables numéricas y booleanas del backend se validan al iniciar; una configuración inválida impide el arranque.

Cambiar una contraseña en .env no rota automáticamente un login que ya existe dentro del volumen. El DDL solo crea logins ausentes. En una instalación existente se conservan las claves originales o se realiza una rotación administrativa coordinada. No se borra el volumen para corregir una contraseña.

6. Arranque de SQL Server con Docker

6.1. Crear los servicios

```
docker compose up -d
docker compose ps -a
docker compose logs db-init
```

Se crean dos servicios. sqlserver mantiene el motor y los datos; db-init es una tarea que ejecuta init.sql cuando el motor está saludable. Es normal que db-init quede detenido después de finalizar: debe terminar con código 0. Comprobar ambos resultados; la salud del motor por sí sola no demuestra que el esquema esté listo.

La imagen de SQL Server 2022 está fijada por digest en Compose. La edición Developer se utiliza para desarrollo y evaluación. El puerto del contenedor se publica únicamente en 127.0.0.1. Los montajes del DDL son de solo lectura; la opción z permite el etiquetado compartido en sistemas con SELinux.

6.2. Qué hace la inicialización

1. Verifica la contraseña de los dos usuarios de aplicación.
2. Ejecuta sqlcmd con autenticación administrativa, confianza en el certificado local y salida con error si falla un lote.
3. Crea ComercialLaEstrella si no existe y crea las tablas ausentes.
4. Define restricciones, índices y la tabla de control de importación.
5. Crea o actualiza vistas y triggers.
6. Crea los logins y usuarios ausentes y concede permisos por objeto.

La inicialización crea estructura; no carga registros de negocio. El histórico se envía por HTTP. Si se necesita volver a aplicar el script sin eliminar los datos:

```
docker compose run --rm db-init
```

El script permite repetir su ejecución sobre el esquema esperado. No reemplaza una herramienta general de migraciones: las tablas existentes no se reconstruyen automáticamente cuando cambia su definición.

6.3. Persistencia y detención

```
# Detener o volver a iniciar sin borrar datos
docker compose stop
docker compose start

# Reiniciar solamente el motor
docker compose restart sqlserver
```

Los archivos se guardan en el volumen sqlserver-data del proyecto Compose. docker compose down elimina los contenedores y la red, pero conserva el volumen. La opción down -v también elimina el volumen y sus datos; no se utiliza en el reinicio habitual ni antes de la demostración.

7. Compilación y ejecución de la API

```
npm run build
npm start
```

build compila TypeScript a dist; start ejecuta dist/server.js. La terminal queda ocupada mientras se atienden peticiones. Para trabajar con recarga automática puede utilizarse npm run dev. Si se modifica el código y se usa npm start, primero se recompila; editar TypeScript no cambia un compilado previo por sí solo.

El servidor inicializa primero el pool de base de datos. Solo después abre el puerto HTTP. Al recibir SIGINT o SIGTERM cierra el servidor y las conexiones, evitando dejar el proceso de Node activo innecesariamente.

```
curl http://127.0.0.1:3000/api/health
```

Resultado esperado: HTTP 200 y el siguiente objeto:

```
{"estado":"ok","database":"SQL Server 2022"}
```

La comprobación ejecuta SELECT 1 contra el motor; confirma comunicación con SQL Server, no solo que Express esté escuchando.

7.1. Arranque diario con la carga ya realizada

1. Abrir Proyecto 2 y ejecutar docker compose up -d.
2. Verificar motor saludable e inicialización correcta.
3. Mantener IMPORT_ENABLED=false e iniciar npm start.
4. Consultar health y el estado de importación.
5. Seleccionar el entorno local de Postman y comenzar una nueva ejecución.

No es necesario importar de nuevo, reinstalar dependencias ni recrear la base para cada sesión. Reiniciar Node tampoco cambia los identificadores que Postman conserva.

8. Migración del modelo y del histórico

8.1. Cambios entre Oracle y SQL Server

La fase 2 mantiene las entidades, claves y dependencias de la fase 1. Los tipos se adaptan al motor: NUMBER de identificadores pasa a INT; importes a DECIMAL(12,2); VARCHAR2 a NVARCHAR para conservar Unicode; y fechas a DATETIME2(0). No se utiliza IDENTITY porque los identificadores originales deben preservarse.

persona concentra los datos compartidos de clientes y empleados. catalogo_producto representa el par tienda-producto y almacena precio vigente y existencia por tienda. detalle_venta conserva el precio histórico aplicado a una venta; este precio puede diferir del vigente sin que exista un error. La FK compuesta de venta hacia empleado garantiza que el empleado pertenece a la tienda indicada.

8.2. Conteos de referencia

La fuente es el libro dataset_comercial_la_estrella.xlsx de la fase 1. data/historico.json contiene su conversión y data/manifest.json registra el origen y los conteos. Los 45 correos ausentes se conservan como NULL; un índice único filtrado excluye los nulos y mantiene la unicidad de los correos presentes.

Tabla	Filas históricas	Clave primaria
pais	3	id_pais
departamento	32	id_departamento
municipio	80	id_municipio
tipo_tienda	4	id_tipo_tienda
tipo_identificacion	3	id_tipo_identificacion
cargo	4	id_cargo
categoria	10	id_categoria
marca	15	id_marca
estado_venta	3	id_estado_venta
metodo_pago	4	id_metodo_pago
persona	388	id_persona
tienda	20	id_tienda
empleado	88	id_empleado
cliente	300	id_cliente
producto	114	id_producto
catalogo_producto	1,424	id_tienda, id_producto
venta	1,500	id_venta

Tabla	Filas históricas	Clave primaria
`detalle_venta`	3,687	`id_venta`, `id_producto`
`pago`	1,389	`id_pago`
TOTAL	**9,068**	19 tablas de negocio

8.3. Primera importación: solo sobre una instalación vacía

1. Detener la API y cambiar temporalmente IMPORT_ENABLED a true en .env.
2. Iniciar la API y, desde otra terminal, ejecutar npm run import:historico.
3. Comprobar HTTP 201, el mensaje Historico importado y conciliado y total 9068.
4. Detener la API, restaurar IMPORT_ENABLED=false y volver a iniciarla.
5. Consultar GET /api/admin/importacion; debe indicar importado=true, detalle.filas=9068 y habilitada=false.

El script cliente envía el archivo mediante POST /api/admin/importacion con X-Import-Key. No escribe directamente en SQL. La colección de importación ofrece una alternativa manual: definir importKey localmente, elegir el archivo histórico en Body, modo binary, y enviar Content-Type application/json. No exportar el secreto.

8.4. Transacción, seguridad y conciliación

1. El servicio reserva una exclusión en memoria antes de su primera operación asíncrona.
2. Compara el SHA-256 canónico del cuerpo con el del archivo histórico incluido. Cambiar el orden de propiedades no cambia la representación canónica; alterar valores o el orden de las filas sí puede cambiarla.
3. Valida cada registro con las reglas de su entidad.
4. Abre el pool estrella_import e inicia una transacción SERIALIZABLE.
5. Adquiere un bloqueo de aplicación exclusivo con sp_getapplock y verifica ausencia de carga previa y de filas en las 19 tablas.
6. Activa un contexto de sesión de importación y carga en orden de dependencias, en bloques de 50 filas.
7. Vuelve a leer y compara todos los valores de cada tabla ordenados por clave; normaliza las fechas para una comparación estable.
8. Registra la carga en importacion_historica, limpia el contexto y confirma. Ante cualquier fallo revierte la transacción.
9. Libera la conexión, destruye el pool especial y libera la exclusión, incluso cuando hay un error.

La excepción histórica requiere simultáneamente pertenecer al rol importador y tener el contexto activo. Esto evita volver a descontar existencias que ya representan el estado histórico del Excel. Las PK, FK, CHECK e índices únicos siguen evaluándose. El usuario estrella_api no pertenece a ese rol y no puede activar la excepción únicamente fijando el contexto.

Hash registrado en la evidencia de carga:

```
80e4d18ae9300bcedbdc1354eddb17b29d2415e053085a9e05d0fae6462882ab
```

Nota. El contador 9068 describe la importación original. Las pruebas posteriores crean registros adicionales; el total actual de las tablas puede ser mayor sin invalidar la conciliación histórica. No borrar la base para hacer coincidir esos números.

9. Contrato HTTP, validación y errores

La dirección base es `http://127.0.0.1:3000/api`. Los cuerpos POST y PUT se envían como JSON con Content-Type application/json. Los identificadores se proporcionan explícitamente; no los asigna automáticamente el servidor.

Método	Ruta de ejemplo	Respuesta
POST	/api/pais	201 y registro creado.
GET	/api/pais?limit=5&offset=0	200 con data, total, limit y offset.
GET	/api/pais/1	200 con objeto; 404 si no existe.
PUT	/api/pais/1	200 con registro actualizado.
DELETE	/api/pais/1	204, sin cuerpo de respuesta.

PUT admite uno o más campos modificables: no obliga a reenviar el registro completo. No admite claves primarias, campos desconocidos ni un objeto vacío. En pago tampoco se admite trasladar el registro a otra venta mediante id_venta.

9.1. Listados y paginación

limit es un entero de 1 a 500, con valor predeterminado 100; offset es un entero no negativo, con valor predeterminado 0. El listado se ordena por clave primaria. Los parámetros no reconocidos se rechazan. total representa el número total de registros de esa entidad, no solo los de la página.

```
{
  "data": [{"id_pais": 1, "nombre": "Guatemala"}],
  "total": 3,
  "limit": 1,
  "offset": 0
}
```

El total del ejemplo corresponde al histórico de países; puede aumentar si se dejan registros de prueba.

9.2. Tipos y restricciones de entrada

- Los enteros se envían como números JSON, no como cadenas. IDs y cantidades deben ser positivos y caber en INT; existencia_actual permite cero.
- Los importes deben ser positivos, con máximo dos decimales y dentro de DECIMAL(12,2). El subtotal debe coincidir con cantidad por precio.
- Los textos tienen longitud máxima según entidad y no pueden contener únicamente espacios. correo admite NULL o una dirección válida.
- Las fechas aceptan YYYY-MM-DD o YYYY-MM-DDTHH:mm:ss. Se interpretan en UTC; en las respuestas pueden aparecer con Z y milisegundos. La entrada no utiliza sufijo Z porque el validador espera los formatos indicados.
- Las claves de ruta deben ser enteros positivos sin signos ni decimales. Para claves compuestas se respetan los dos segmentos y su orden.

9.3. Códigos HTTP y lectura de fallos

Código	Significado en esta API
200	Consulta o actualización correcta.
201	Creación, lote o importación correcta.
204	Eliminación correcta; no hay JSON que mostrar.
400	Datos o JSON inválidos, histórico alterado o stock insuficiente.
401	Clave de importación incorrecta con la ruta habilitada.
404	Registro o ruta inexistente; importación deshabilitada.
409	Duplicado, referencias, operación protegida, carga previa o conflicto concurrente.
413	Cuerpo de la petición superior al tamaño permitido.
500	Error interno no clasificado; revisar terminal del servidor.

Los mensajes de negocio se devuelven en error; la validación también puede incluir detalles con campo y mensaje. Los números SQL 2601 y 2627 se traducen a Registro duplicado; 547 a conflicto de integridad; 1205 y 1222 a conflicto concurrente. El error 51001 de los triggers se convierte en 400 y Stock insuficiente; el 51002 se comunica como 409. No se exponen consultas completas ni contraseñas en las respuestas de error interno.

10. TypeORM y acceso a datos

Los metadatos de src/entities/schema.json describen tabla, claves, tipos, longitudes y nulabilidad. src/entities/index.ts los valida y crea un EntitySchema por tabla, con esquema dbo. Las relaciones físicas y restricciones se mantienen en init.sql. No se duplican mediante sincronización automática del ORM.

- synchronize=false y migrationsRun=false: el esquema lo administra el DDL explícito.
- El pool normal permite hasta cinco conexiones; el de importación, una. No se abre una sesión nueva por cada petición.
- encrypt=true y trustServerCertificate=true permiten cifrado con el certificado local del contenedor. Esa confianza se ha definido para la instalación local.
- useUTC=true conserva una interpretación uniforme de fechas; requestTimeout es 120000 ms.
- disableOutputReturning=true evita OUTPUT sin INTO, incompatible con escrituras en tablas con triggers habilitados. Después de insertar o actualizar se consulta el resultado.

Las rutas de entidades llaman a una fábrica de CRUD; así conservan módulos separados y comparten el contrato de validación. Los nombres de tabla proceden de metadatos internos, nunca de un nombre arbitrario recibido por HTTP. Los valores se pasan como parámetros o mediante operaciones de repositorio.

11. Inventario: trigger Insert_Detalle

11.1. Qué es un trigger

Un trigger DML es código almacenado que SQL Server ejecuta automáticamente como respuesta a INSERT, UPDATE o DELETE sobre una tabla. El cliente no lo invoca con una ruta especial. Forma parte de la transacción que modificó la tabla: si lanza un error, no se debe conservar una parte de la venta y otra parte del inventario.

Insert_Detalle se define como AFTER INSERT, UPDATE, DELETE. Se ejecuta una vez por sentencia y trabaja con los conjuntos inserted y deleted. En INSERT solo hay filas nuevas; en DELETE solo anteriores; en UPDATE hay ambas versiones. No se presupone una sola fila.

11.2. Cálculo del ajuste

El trigger reúne las ventas afectadas, bloquea sus encabezados, comprueba que estén REGISTRADA y sin pagos, y agrupa por tienda y producto:

$$\Delta_{t,p} = \sum \text{cantidad nueva}_{t,p} - \sum \text{cantidad anterior}_{t,p} \qquad \text{stock final}_{t,p} = \text{stock actual}_{t,p} - \Delta_{t,p}$$

En una inserción de 2 unidades el delta es 2 y se descuentan 2. Si se modifica de 2 a 3, el delta es 1: solo se descuenta una unidad adicional. Si se elimina un detalle de 3, el delta es -3 y se devuelven tres.

11.3. Actualización atómica y rollback

La actualización del catálogo utiliza UPDLOCK y HOLDLOCK, junto con una condición que exige stock suficiente. Después compara las filas realmente actualizadas con el número de grupos del delta. Si falta un catálogo o alguna cantidad supera el disponible, lanza THROW 51001. SET XACT_ABORT ON y el error impiden conservar ajustes parciales.

En un lote que solicita 4 y 7 unidades del mismo producto en una tienda con 10, se considera el total 11. No se aceptan ambas filas por separado. En competencia concurrente, las solicitudes no deben consumir simultáneamente las mismas últimas unidades; los bloqueos y las condiciones se evalúan en el motor. Un conflicto de bloqueo puede responder 409 y requerir reintento.

11.4. Demostración mínima

Crear un producto de prueba, su catálogo en tienda 1 con existencia 10 y una venta REGISTRADA. Pedir 11 unidades debe devolver 400, sin crear el detalle y conservando 10. Pedir 2 debe devolver 201; consultar el catálogo debe mostrar 8. Los dos resultados se verifican por HTTP, sin modificar directamente la tabla.

12. Pagos: trigger Insert_Pago y estados

Insert_Pago se ejecuta después de INSERT, UPDATE y DELETE en pago. Reúne ventas afectadas, toma bloqueos sobre los encabezados, rechaza ventas anuladas y evita recibir pagos si no existen detalles.

$$T = \sum \text{subtotal de detalles} \qquad P = \sum \text{monto de pagos}$$

Si T es mayor que cero y P es mayor o igual a T, el estado pasa a PAGADA; en otro caso, a REGISTRADA. La implementación acepta sobrepago y puede mostrar un saldo negativo: no debe afirmarse que lo prohíbe. Actualizar o eliminar un abono recalcula el estado con los importes restantes.

12.1. Ejemplo de venta de Q50

1. Registrar dos unidades con precio Q25 y subtotal Q50.
2. Registrar un primer pago de Q20; GET de la venta muestra REGISTRADA, total 50, pagado 20 y saldo 30.
3. Registrar un segundo pago de Q30; el motor cambia a PAGADA, total pagado 50 y saldo 0.
4. Reducir o eliminar el segundo pago en una prueba separada permite observar el regreso a REGISTRADA si el total pagado deja de cubrir la venta.

El backend no asigna PAGADA como parte del POST de pago. SQL Server realiza la transición; el GET posterior permite observar su resultado.

13. Protecciones complementarias del negocio

13.1. Validar_Venta

Una venta nueva debe comenzar REGISTRADA. No se permite marcar PAGADA manualmente. El trigger reconoce la actualización anidada originada por Insert_Pago para permitir el cambio interno válido. Cuando una venta ya tiene detalles o pagos se protegen tienda, empleado, cliente y fecha; cambiar esos valores alteraría el significado de movimientos existentes.

Una venta REGISTRADA sin pagos puede cambiar a ANULADA. La transición agrupa sus detalles y devuelve existencias al catálogo una sola vez. Repetir la anulación se rechaza, por lo que no duplica la devolución. Una venta anulada queda protegida. Las FK impiden borrar ventas con registros asociados; no hay una eliminación en cascada de transacciones financieras.

13.2. Proteger_Catalogo y Proteger_Estados

Proteger_Catalogo impide eliminar una combinación tienda-producto si existen detalles de ventas que necesitan ese contexto. Permite actualizar precio o existencias; no elimina el registro histórico del detalle.

Proteger_Estados conserva el contrato fijo: 1=REGISTRADA, 2=PAGADA y 3=ANULADA. Los módulos CRUD existen para esa entidad, pero las escrituras incompatibles se rechazan con 409. Su prueba correcta es demostrar la protección, no cambiar los nombres del sistema.

13.3. Resumen de dependencias

Las altas respetan la secuencia de catálogos, personas, tiendas, empleados/clientes, productos, catálogo, ventas, detalles y pagos. Para borrar auxiliares se usa el orden inverso. Un 409 por FK significa que existe una referencia que debe tratarse conforme al negocio; no se desactiva la restricción para forzar la operación.

14. Vistas y reportes SQL

Una vista guarda una definición SELECT reutilizable. Las tres vistas solicitadas encapsulan JOIN y agregaciones en el motor. Los endpoints de reportes ejecutan SELECT sobre esos objetos y aplican un orden estable; no reconstruyen los reportes mediante relaciones del ORM.

14.1. Vista_Ventas_Ubicacion

Ruta: `GET/api/reportes/ventas-ubicacion`. Devuelve id_tienda, tienda, municipio, departamento, pais, cantidad_ventas y total_facturado. Preagrupa los subtotales por venta antes de contar y sumar, y excluye ANULADA. Parte de las ventas existentes; una tienda sin ventas no aparece por el solo hecho de existir.

14.2. Vista_Top_Productos

Ruta: `GET/api/reportes/top-productos`. Devuelve codigo_producto, nombre, categoria, marca, unidades_vendidas y monto_total. Agrega detalles de ventas no anuladas y usa BIGINT en la suma de unidades. El endpoint ordena por unidades descendentes y código de producto como desempate. El reporte no está limitado a los primeros diez: devuelve todos los productos presentes en esa agregación.

14.3. Vista_Saldos_Pendientes y vista auxiliar

Ruta: `GET/api/reportes/saldos-pendientes`. Devuelve id_venta, total_venta, total_pagado y saldo_pendiente para ventas REGISTRADA. Se apoya en Vista_Consulta_Ventas, que también enriquece los GET de venta con estado e importes. COALESCE convierte la ausencia de detalles o abonos en cero.

14.4. Por qué se agregan detalles y pagos por separado

Si una venta tiene dos detalles y tres pagos, unir ambos conjuntos directamente genera seis filas. Eso repetiría los subtotales tres veces y los pagos dos veces. Las expresiones de tabla comunes agrupan cada conjunto por id_venta antes de unirlos al encabezado. El saldo se calcula como total de detalles menos total de pagos.

Todos los endpoints de reportes responden HTTP 200 con un objeto que contiene data como arreglo. En los resultados del controlador mssql, ciertos agregados BIGINT pueden representarse como cadenas JSON; por ejemplo, cantidad_ventas o unidades_vendidas. Los totales pueden cambiar después de las pruebas; una cifra fija de una captura no sustituye la comprobación de la fórmula.

15. Postman: importación, variables y ejecución

15.1. Importar los archivos correctos

En Postman seleccionar Import, Files y abrir los siguientes archivos de la carpeta postman:

- ComercialLaEstrella.postman_collection.json: crea la colección Comercial La Estrella - Fase 2.
- local.postman_environment.json: crea el entorno Comercial La Estrella local.

Seleccionar el entorno arriba a la derecha. La colección aparece en Collections y el entorno en Environments. Importar JSON no ejecuta peticiones ni crea tablas o registros.

Carpeta	Uso
`00 Preparacion`	Comprueba conexión, genera IDs y crea dependencias auxiliares.
`01 CRUD por entidad`	Prueba cada entidad; contiene 19 subcarpetas.
`02 Limpieza auxiliares`	Elimina las dependencias temporales del CRUD.
`03 Triggers`	Demuestra inventario, pagos y anulación con resultados observables.
`04 Vistas`	Consulta los tres reportes.

15.2. Qué significan las llaves dobles

Una expresión como `{{baseUrl}}` es una referencia a una variable. Antes de enviar, Postman sustituye su valor. baseUrl está en el entorno local y también como respaldo en la colección. Su valor predeterminado es http://127.0.0.1:3000/api.

runId, auxId y payId son variables de colección creadas por el script Pre-request de la primera petición, Iniciar ejecucion y comprobar conexion. Se pueden inspeccionar en Scripts de esa petición y en Variables de la colección después de ejecutarla.

```
const id = 1500000000 + Math.floor(Math.random() * 100000000);
pm.collectionVariables.set('runId', id);
pm.collectionVariables.set('auxId', id + 1);
pm.collectionVariables.set('payId', id + 2);
```

El rango aleatorio reduce la probabilidad de colisión, pero no garantiza unicidad absoluta. Si se vuelve a enviar Crear con el mismo runId, el primer POST puede dar 201 y el segundo 409. Esto es esperado porque se intenta repetir la PK. Si un ID nuevo coincide con uno existente, volver a iniciar la preparación antes de comenzar otra ejecución.

15.3. Cómo repetir mañana la demostración

Ejecutar una vez la primera petición de preparación y continuar en el orden original. Esa petición genera nuevos IDs; no borra datos ni reinicia la base. Reiniciar Node o Docker no cambia las variables de Postman. No volver a ejecutar la primera petición a mitad de un CRUD: se perdería la referencia al registro que se pretendía consultar o eliminar.

La forma más reproducible es hacer clic derecho en la colección, Run collection, seleccionar todas las carpetas y ejecutar una iteración. El orden evita fallos por dependencias. Para pruebas manuales se utiliza Send y se conserva el mismo runId durante toda la secuencia. Test Results muestra las aserciones aprobadas o fallidas. Un código 400, 404 o 409 puede ser precisamente lo que la prueba espera.

15.4. Ejemplo manual de CRUD

Ejecutar primero la preparación. En pais: Crear devuelve 201, Listar 200, Buscar por clave 200, Actualizar 200, Eliminar 204 y Confirmar eliminacion 404. Tras la eliminación, el 404 demuestra que el registro ya no está. Los nombres Demo y los IDs altos permiten distinguir estos registros del histórico.

15.5. Colección de importación

Importacion.postman_collection.json se reserva para una base vacía con la ruta de carga habilitada. No se ejecuta como parte de la demostración habitual. Si el histórico ya está cargado, basta con consultar su estado. El backend impide repetir la carga; no existe una petición de reinicio de base incluida en la colección.

16. Lotes, atomicidad y concurrencia

Las rutas POST /api/detalle_venta/lote y POST /api/pago/lote admiten arreglos de 1 a 100 objetos con los mismos campos del POST individual. Devuelven 201 con insertados igual a la cantidad de filas. La inserción es una sola sentencia; el trigger considera el conjunto completo y cualquier error revierte el lote.

Atomicidad significa que se conserva todo el cambio o ninguno. La prueba de lotes debe incluir un caso cuyo consumo agregado exceda el stock y verificar que no permanezcan detalles ni descuentos parciales. La prueba concurrente lanza solicitudes que compiten por las últimas unidades y comprueba que no se sobrevende. No se concluye concurrencia correcta únicamente porque dos peticiones separadas funcionen en serie.

17. Pruebas y resultados registrados

17.1. Comandos reproducibles

```
npm run build
npm test
# API encendida e historico cargado:
npm run test:integration
npm run postman
```

Las pruebas unitarias no necesitan la API levantada. Las de integración y Postman escriben registros reales mediante HTTP y requieren el histórico. No son una simulación sin efectos. El CLI oficial de Postman se incluye como dependencia de desarrollo; no depende de una instalación gráfica para ejecutar la colección.

Comprobación registrada	Resultado y alcance
Carga histórica	9068 filas; todos los campos de las 19 tablas conciliados antes del commit.
Pruebas unitarias	4 correctas: validación de entradas y exclusión de importación.
Integración HTTP	8 escenarios correctos; el ejecutor también cuenta el test padre.
Postman CLI oficial	144 peticiones y 155 aserciones; 0 fallos.
Persistencia	Estado de importación y venta PAGADA idénticos antes y después de reiniciar el motor.
Dependencias	La evidencia guardada registra 0 vulnerabilidades en la auditoría de esa ejecución.

Estos números proceden de docs/evidencias/validacion.json y sus registros asociados; describen la ejecución guardada, no una garantía sobre cambios futuros. Los archivos registran fechas UTC del 17 de octubre, que pueden corresponder todavía al 16 de octubre en Guatemala. Para un resultado actualizado se vuelven a ejecutar las comprobaciones.

17.2. Escenarios de integración

1. Estado del histórico protegido y conciliado.
2. CRUD de catálogos y entidades relacionadas, incluyendo conflictos.
3. Stock insuficiente, rollback y ajustes por UPDATE y DELETE.
4. Abonos parciales, sobrepago, correcciones y protecciones.
5. Lotes con consumo agregado, rollback total y varios pagos.
6. Competencia concurrente por las últimas unidades.
7. Anulación con devolución única e impacto en reportes.
8. Comparación de las tres vistas con una referencia calculada desde los datos obtenidos por la API.

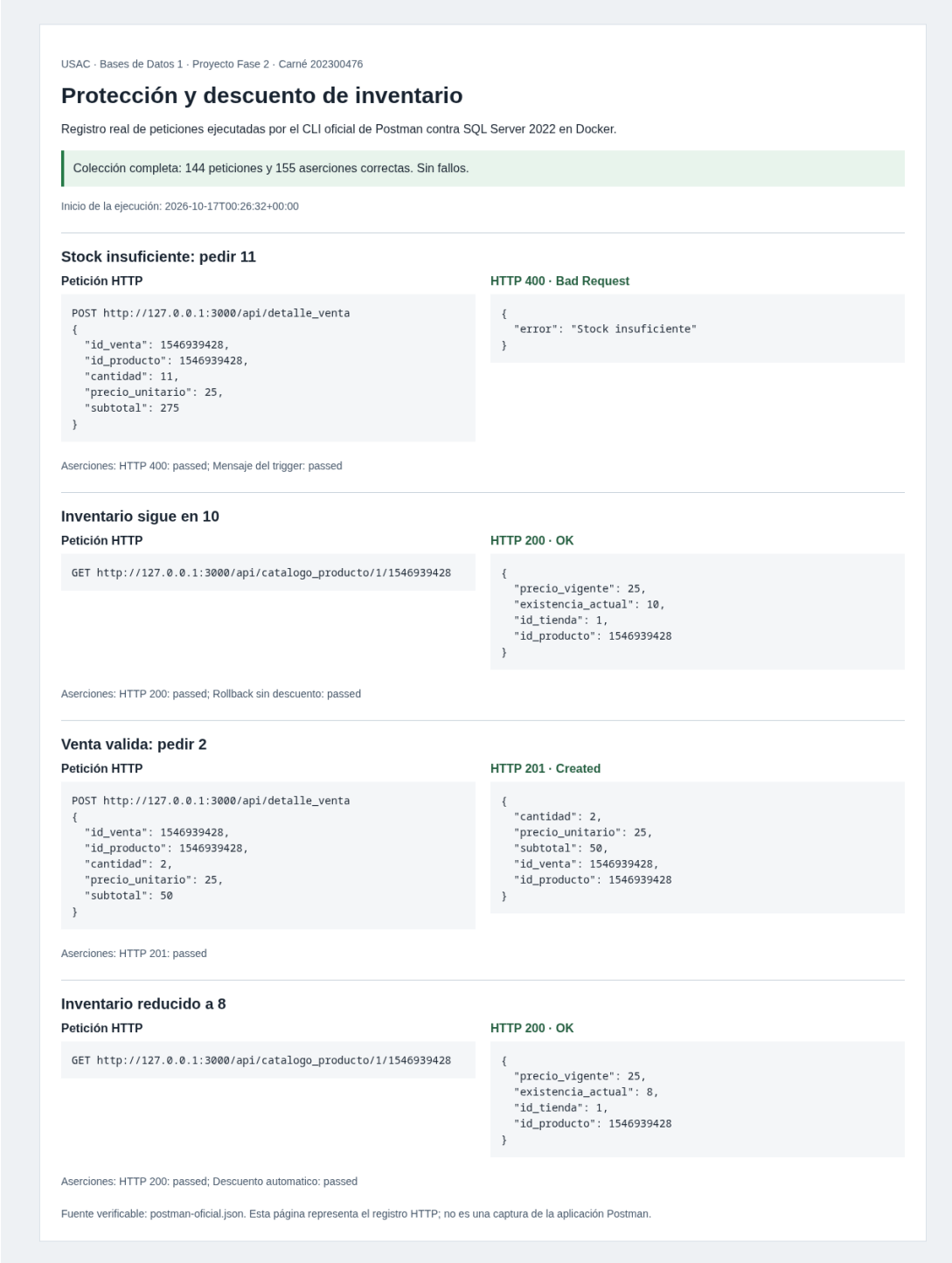
En las evidencias de integración, los listados grandes se guardan como muestras identificadas. No deben interpretarse como si solo existieran las filas visibles en la muestra. La comprobación automatizada consulta las páginas necesarias.

18. Evidencias de inventario y pagos

Las imágenes siguientes se generaron a partir de respuestas reales guardadas por el CLI oficial de Postman. Son capturas de una representación HTML del registro HTTP, no capturas de la interfaz gráfica de Postman. Las respuestas y aserciones originales están en postman-oficial.json; los HTML permiten inspeccionar el mismo contenido con un navegador.

18.1. Inventario: rechazo, conservación y descuento

La secuencia muestra stock inicial 10, intento inválido de 11, respuesta Stock insuficiente y conservación del inventario. A continuación, una venta de 2 unidades produce stock 8. La evidencia combina la respuesta del POST y el GET posterior para comprobar el efecto en la base.



USAC · Bases de Datos 1 · Proyecto Fase 2 · Carné 202300476

Protección y descuento de inventario

Registro real de peticiones ejecutadas por el CLI oficial de Postman contra SQL Server 2022 en Docker.

Colección completa: 144 peticiones y 155 aserciones correctas. Sin fallos.

Inicio de la ejecución: 2026-10-17T00:26:32+00:00

Stock insuficiente: pedir 11

Petición HTTP

```
POST http://127.0.0.1:3000/api/detalle_venta
{
  "id_venta": 1546939428,
  "id_producto": 1546939428,
  "cantidad": 11,
  "precio_unitario": 25,
  "subtotal": 275
}
```

HTTP 400 · Bad Request

```
{
  "error": "Stock insuficiente"
}
```

Aserciones: HTTP 400: passed; Mensaje del trigger: passed

Inventario sigue en 10

Petición HTTP

```
GET http://127.0.0.1:3000/api/catalogo_producto/1/1546939428
```

HTTP 200 · OK

```
{
  "precio_vigente": 25,
  "existencia_actual": 10,
  "id_tienda": 1,
  "id_producto": 1546939428
}
```

Aserciones: HTTP 200: passed; Rollback sin descuento: passed

Venta valida: pedir 2

Petición HTTP

```
POST http://127.0.0.1:3000/api/detalle_venta
{
  "id_venta": 1546939428,
  "id_producto": 1546939428,
  "cantidad": 2,
  "precio_unitario": 25,
  "subtotal": 50
}
```

HTTP 201 · Created

```
{
  "cantidad": 2,
  "precio_unitario": 25,
  "subtotal": 50,
  "id_venta": 1546939428,
  "id_producto": 1546939428
}
```

Aserciones: HTTP 201: passed

Inventario reducido a 8

Petición HTTP

```
GET http://127.0.0.1:3000/api/catalogo_producto/1/1546939428
```

HTTP 200 · OK

```
{
  "precio_vigente": 25,
  "existencia_actual": 8,
  "id_tienda": 1,
  "id_producto": 1546939428
}
```

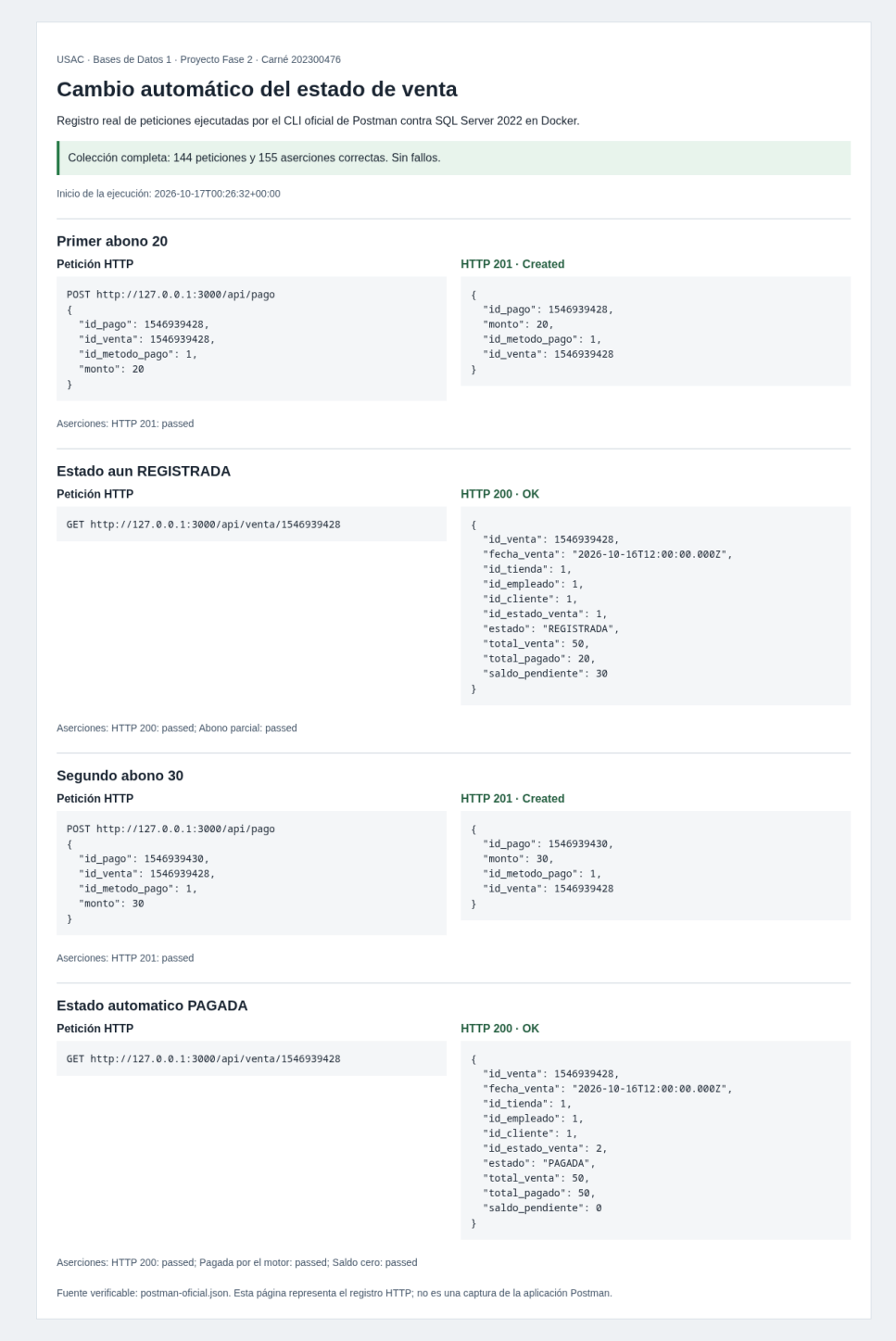
Aserciones: HTTP 200: passed; Descuento automatico: passed

Fuente verificable: postman-oficial.json. Esta página representa el registro HTTP; no es una captura de la aplicación Postman.

Figura 1: Respuestas reales del CLI de Postman: error de stock, rollback y descuento válido. Representación del registro HTTP.

18.2. Abonos y cambio automático de estado

La segunda secuencia muestra una venta de total 50, un abono parcial de 20 y el abono restante de 30. El GET de la venta permite comprobar REGISTRADA antes de completar el pago y PAGADA con saldo cero después.



USAC · Bases de Datos 1 · Proyecto Fase 2 · Carné 202300476

Cambio automático del estado de venta

Registro real de peticiones ejecutadas por el CLI oficial de Postman contra SQL Server 2022 en Docker.

Colección completa: 144 peticiones y 155 aserciones correctas. Sin fallos.

Inicio de la ejecución: 2026-10-17T00:26:32+00:00

Primer abono 20

Petición HTTP

```
POST http://127.0.0.1:3000/api/pago
{
  "id_pago": 1546939428,
  "id_venta": 1546939428,
  "id_metodo_pago": 1,
  "monto": 20
}
```

HTTP 201 · Created

```
{
  "id_pago": 1546939428,
  "monto": 20,
  "id_metodo_pago": 1,
  "id_venta": 1546939428
}
```

Aserciones: HTTP 201: passed

Estado aun REGISTRADA

Petición HTTP

```
GET http://127.0.0.1:3000/api/venta/1546939428
```

HTTP 200 · OK

```
{
  "id_venta": 1546939428,
  "fecha_venta": "2026-10-16T12:00:00.000Z",
  "id_tienda": 1,
  "id_empleado": 1,
  "id_cliente": 1,
  "id_estado_venta": 1,
  "estado": "REGISTRADA",
  "total_venta": 50,
  "total_pagado": 20,
  "saldo_pendiente": 30
}
```

Aserciones: HTTP 200: passed; Abono parcial: passed

Segundo abono 30

Petición HTTP

```
POST http://127.0.0.1:3000/api/pago
{
  "id_pago": 1546939430,
  "id_venta": 1546939428,
  "id_metodo_pago": 1,
  "monto": 30
}
```

HTTP 201 · Created

```
{
  "id_pago": 1546939430,
  "monto": 30,
  "id_metodo_pago": 1,
  "id_venta": 1546939428
}
```

Aserciones: HTTP 201: passed

Estado automatico PAGADA

Petición HTTP

```
GET http://127.0.0.1:3000/api/venta/1546939428
```

HTTP 200 · OK

```
{
  "id_venta": 1546939428,
  "fecha_venta": "2026-10-16T12:00:00.000Z",
  "id_tienda": 1,
  "id_empleado": 1,
  "id_cliente": 1,
  "id_estado_venta": 2,
  "estado": "PAGADA",
  "total_venta": 50,
  "total_pagado": 50,
  "saldo_pendiente": 0
}
```

Aserciones: HTTP 200: passed; Pagada por el motor: passed; Saldo cero: passed

Fuente verificable: postman-oficial.json. Esta página representa el registro HTTP; no es una captura de la aplicación Postman.

Figura 2: Respuestas reales del CLI de Postman: pagos parciales y transición automática a PAGADA. Representación del registro HTTP.

18.3. Comprobación de persistencia

El archivo persistencia.json conserva el estado del histórico y la misma venta pagada antes y después de reiniciar SQL Server. El id de la venta de esa evidencia es 1546939428; total y pagado permanecen en 50 y saldo en 0. Es un ejemplo de la ejecución registrada, no el ID de cada nueva demostración.

Para reproducirla, consultar el estado de importación y una venta de prueba, guardar las respuestas, reiniciar únicamente sqlserver, esperar su disponibilidad y repetir los GET. Comparar identificadores, estados, importes y hash. Esta prueba verifica el volumen; no requiere borrar ni volver a importar nada.

19. Inspección estructural con DBeaver

Crear una conexión SQL Server hacia 127.0.0.1, puerto 1433 y base ComercialLaEstrella, usando las credenciales DB_USER y DB_PASSWORD de .env. Para el certificado local se utiliza encrypt=true y trustServerCertificate=true. No copiar las contraseñas al manual ni a capturas públicas.

En la instalación local se guardó la conexión Comercial La Estrella. Expandir base, esquemas, dbo y tablas para mostrar su estructura; las vistas y triggers se inspeccionan en sus nodos correspondientes. El modo de solo lectura del cliente ayuda a evitar cambios accidentales, pero no convierte por sí mismo al login de la API en un usuario SQL con permisos exclusivamente SELECT.

Durante la evaluación no ejecutar INSERT, UPDATE ni DELETE directamente desde DBeaver. Las operaciones de negocio se muestran en Postman y las consultas de estructura se mantienen separadas de las pruebas CRUD.

20. Guion detallado para la demostración

1. Mostrar el repositorio y ubicar Compose e init.sql en la raíz del proyecto. Explicar que SQL Server se ejecuta dentro de Docker y la API en Node en el equipo.
2. Mostrar docker compose ps -a y el resultado de db-init. Identificar motor saludable, inicializador terminado correctamente y volumen persistente.
3. Ejecutar health y consultar admin/importacion. Explicar los 9068 registros históricos y por qué los conteos actuales pueden incluir pruebas.
4. Seleccionar el entorno local e iniciar una sola vez la preparación. Explicar baseUrl y los IDs generados por el script.
5. Demostrar CRUD de pais. Leer el cuerpo de creación, consultar, actualizar, eliminar y justificar el 404 final.
6. Mostrar una ruta de clave compuesta y explicar por qué catálogo necesita tienda más producto y detalle necesita venta más producto.
7. Ejecutar las pruebas de inventario. Justificar 400 al pedir 11 con stock 10 y el GET que confirma 10; luego vender 2 y comprobar 8.
8. Ejecutar abonos 20 y 30 de la venta de 50. Mostrar el GET entre ambos y el estado PAGADA final; ubicar Insert_Pago en SQL.
9. Mostrar anulación sin pagos y devolución única. Explicar por qué repetirla se rechaza.
10. Ejecutar las tres vistas y explicar las columnas, JOIN, exclusión de anuladas y preagregación.
11. Abrir Test Results o el resumen del runner; interpretar fallos esperados frente a aserciones fallidas.
12. Mostrar el PDF, la colección exportada y el código. Realizar cualquier ejercicio práctico con validación y una prueba HTTP del comportamiento modificado.

20.1. Preparación de una grabación

Antes de grabar, realizar una ejecución completa para confirmar conexión y orden; mantener visibles los códigos y el JSON; ampliar texto si hace falta y ocultar credenciales. Para la toma definitiva iniciar otra preparación y seguir la misma secuencia. No depender de un ID de una sesión anterior ni reiniciar a mitad de una transacción de demostración. Mostrar el resultado del test junto con la respuesta evita interpretar un 400 esperado como un fallo del proyecto.

21. Solución de problemas

Síntoma	Comprobación y acción
ECONNREFUSED en Postman	Verificar npm start, baseUrl, puerto 3000 y health. Mantener abierta la terminal de la API.
La API no inicia por SQL	Revisar motor saludable, logs de db-init y credenciales del volumen. Una contraseña editada en .env no cambia un login existente.
Variables sin resolver	Seleccionar el entorno correcto y ejecutar la primera petición de preparación. Revisar Variables de la colección.
409 al repetir Crear	Ya existe esa clave. No repetir el POST sobre el mismo ID salvo para demostrar el conflicto; comenzar una ejecución nueva desde preparación.
404 después de eliminar	Es el resultado esperado de Confirmar eliminacion. Revisar la aserción correspondiente.
409 al borrar una entidad	Tiene referencias o una protección de negocio. Eliminar auxiliares en orden inverso y no desactivar FK.
400 por stock	Consultar el catálogo de la tienda correcta; comprobar cantidad y clave compuesta. El intento fallido no debe descontar nada.
400 por fecha o decimal	Usar los formatos de fecha admitidos y números JSON con hasta dos decimales; no enviar símbolos de moneda.
Importación devuelve 404	La ruta está deshabilitada. Si el histórico ya existe, esto es normal y no debe habilitarse para repetirlo.
Código editado sin efecto	Recompilar antes de npm start o utilizar npm run dev para recarga de desarrollo.
Más filas que el manifiesto	Las pruebas agregan datos Demo/QA. Diferenciar el histórico importado de los registros actuales.
db-init aparece Exited	Es normal si terminó con código 0. Si terminó con otro código, revisar su log antes de continuar.

22. Seguridad, límites y entrega reproducible

Los logins de aplicación no son administradores. estrella_api recibe permisos CRUD por tabla y SELECT de vistas/-control; estrella_import tiene permisos para la carga y su registro de control, y pertenece al rol importador. Ninguno necesita crear tablas desde la API. El login sa se reserva para el bootstrap de la infraestructura.

La aplicación no implementa autenticación general de usuarios, autorización por roles HTTP ni exposición pública. Su alcance es evaluación local. La protección de importación combina habilitación temporal, clave de cabecera, archivo permitido, rol SQL y transacción. No es un endpoint de carga arbitraria para cualquier conjunto de datos.

El archivo .env, node_modules, dist, logs y los comprimidos de entrega se excluyen de Git. El ZIP se genera bajo demanda con npm run package y usa el formato exigido por el enunciado, [BD1]_ProyectoFase2_202300476.zip. No hace falta mantenerlo en el repositorio. El script excluye las credenciales y los artefactos de ejecución; quien recibe la entrega genera su propio .env, instala dependencias y compila.

22.1. Documentación editable

La fuente de este manual es docs/Manual_Tecnico_Fase2.tex. El generador scripts/generate-manual.py obtiene los campos de las entidades, ejemplos de Postman, respuestas esperadas y conteos desde los archivos del proyecto. El diseño conserva la portada institucional y la paleta morada y azul del manual de la fase 1.

```
# Desde Proyecto 2: regenerar LaTeX y compilar el PDF
python3 scripts/generate-manual.py --build
```

El PDF final se guarda en docs/output/pdf/Manual_Tecnico_Fase2.pdf. La compilación utiliza un directorio temporal; no deja archivos auxiliares de LaTeX en la entrega. Si se ajusta contenido narrativo generado, editar el generador y regenerar para conservar la reproducibilidad.

23. Preguntas de estudio y respuestas

¿Qué diferencia hay entre API y base de datos?

La API ofrece HTTP y valida la entrada; SQL Server almacena registros y aplica integridad y reglas transaccionales. Postman habla con la API.

¿Por qué TypeORM si existe init.sql?

TypeORM facilita el acceso parametrizado y los repositorios. El DDL explícito permite revisar y reproducir constraints, vistas y triggers sin depender de sincronización automática.

¿Por qué el stock no pertenece al producto?

Una tienda puede tener precio y existencia distintos para el mismo producto; por eso el stock depende de la clave tienda-producto.

¿Qué contienen inserted y deleted?

Son conjuntos de filas de la sentencia que activó el trigger. inserted contiene versiones nuevas y deleted versiones anteriores; UPDATE usa ambos.

¿Quién cambia la venta a PAGADA?

El trigger Insert_Pago en SQL Server compara totales y actualiza el estado. Node comunica el resultado y el GET lo permite observar.

¿Qué es rollback?

Deshacer los cambios de una transacción fallida. Un detalle con stock insuficiente no debe dejar un descuento parcial de inventario.

¿Qué diferencia hay entre PK y FK?

La PK identifica de manera única una fila. La FK exige que la referencia exista en otra clave válida y evita registros huérfanos.

¿Por qué no unir detalles y pagos directamente?

Porque varios detalles y varios pagos se multiplican en el JOIN. Se suman por separado y luego se unen por venta.

¿Qué cambia al reiniciar el backend?

Se reinician el proceso y sus conexiones; los datos permanecen en SQL y las variables de Postman no se regeneran.

¿Qué cambia al ejecutar la primera petición de Postman?

Su Pre-request genera IDs para una nueva prueba y después comprueba health. No borra tablas ni vuelve a cargar el histórico.

¿Por qué se omite el descuento en la carga original?

El dataset ya contiene existencias históricas. Tratar esos detalles como ventas nuevas descontaría dos veces. La excepción es controlada y exclusiva para el archivo original.

¿Una respuesta roja significa error de la prueba?

No necesariamente. Si se está probando un rechazo y la respuesta coincide con lo esperado, la aserción pasa aunque el código HTTP sea 400, 404 o 409.

A. Referencia detallada de las 19 entidades

Las siguientes fichas se obtienen de los metadatos reales y de la colección entregada. Cada una muestra clave, ruta individual, campos, dependencias y cuerpos de ejemplo. Para todas las entidades, POST y GET de listado utilizan /api/ seguido del nombre de tabla; GET individual, PUT y DELETE usan la ruta individual. Los ejemplos con llaves dobles se envían desde Postman después de la preparación. No son JSON independiente hasta que Postman sustituye las variables.

A.1. pais

Catálogo de países para ubicar departamentos.

No depende de otras entidades. El nombre es único.

Ruta individual: `/api/pais/:id_pais`

Campo	Tipo SQL	Contrato de escritura
`id_pais`	`INT`	PK; requerido en POST, inmutable en PUT
`nombre`	`NVARCHAR(80)`	Requerido en POST; modificable en PUT

Cuerpo de POST de la colección:

```
{
  "id_pais": {{runId}},
  "nombre": "Demo pais {{runId}}"
}
```

Cuerpo de PUT de la colección:

```
{
  "nombre": "Demo pais {{runId}}"
}
```

A.2. departamento

División territorial asociada a un país.

id_pais referencia pais. El nombre es único dentro de cada país.

Ruta individual: `/api/departamento/:id_departamento`

Campo	Tipo SQL	Contrato de escritura
`id_departamento`	`INT`	PK; requerido en POST, inmutable en PUT
`nombre`	`NVARCHAR(80)`	Requerido en POST; modificable en PUT
`id_pais`	`INT`	Requerido en POST; modificable en PUT

Cuerpo de POST de la colección:

```
{
  "id_departamento": {{runId}},
  "nombre": "Demo departamento {{runId}}",
  "id_pais": 1
}
```

Cuerpo de PUT de la colección:

```
{
  "nombre": "Demo departamento {{runId}}"
}
```

A.3. municipio

Municipio utilizado por personas y tiendas.

id_departamento referencia departamento. El nombre es único dentro del departamento.

Ruta individual: `/api/municipio/:id_municipio`

Campo	Tipo SQL	Contrato de escritura
`id_municipio`	`INT`	PK; requerido en POST, inmutable en PUT
`nombre`	`NVARCHAR(80)`	Requerido en POST; modificable en PUT
`id_departamento`	`INT`	Requerido en POST; modificable en PUT

Cuerpo de POST de la colección:

```
{
  "id_municipio": {{runId}},
  "nombre": "Demo municipio {{runId}}",
  "id_departamento": 1
}
```

Cuerpo de PUT de la colección:

```
{
  "nombre": "Demo municipio {{runId}}"
}
```

A.4. tipo_tienda

Clasificación comercial de las tiendas.

No depende de otras entidades. El nombre es único.

Ruta individual: `/api/tipo_tienda/:id_tipo_tienda`

Campo	Tipo SQL	Contrato de escritura
`id_tipo_tienda`	`INT`	PK; requerido en POST, inmutable en PUT
`nombre`	`NVARCHAR(80)`	Requerido en POST; modificable en PUT

Cuerpo de POST de la colección:

```
{
  "id_tipo_tienda": {{runId}},
  "nombre": "Demo tipo_tienda {{runId}}"
}
```

Cuerpo de PUT de la colección:

```
{
  "nombre": "Demo tipo_tienda {{runId}}"
}
```

A.5. tipo_identificacion

Tipos de documento de los clientes.

No depende de otras entidades. El nombre es único.

Ruta individual: `/api/tipo_identificacion/:id_tipo_identificacion`

Campo	Tipo SQL	Contrato de escritura
`id_tipo_identificacion`	`INT`	PK; requerido en POST, inmutable en PUT
`nombre`	`NVARCHAR(50)`	Requerido en POST; modificable en PUT

Cuerpo de POST de la colección:

```
{
  "id_tipo_identificacion": {{runId}},
  "nombre": "Demo tipo_identificacion {{runId}}"
}
```

Cuerpo de PUT de la colección:

```
{
  "nombre": "Demo tipo_identificacion {{runId}}"
}
```

A.6. cargo

Catálogo de cargos del personal.

No depende de otras entidades. El nombre es único.

Ruta individual: `/api/cargo/:id_cargo`

Campo	Tipo SQL	Contrato de escritura
`id_cargo`	`INT`	PK; requerido en POST, inmutable en PUT
`nombre`	`NVARCHAR(80)`	Requerido en POST; modificable en PUT

Cuerpo de POST de la colección:

```
{
  "id_cargo": {{runId}},
  "nombre": "Demo cargo {{runId}}"
}
```

Cuerpo de PUT de la colección:

```
{
  "nombre": "Demo cargo {{runId}}"
}
```

A.7. categoria

Categoría de los productos.

No depende de otras entidades. El nombre es único.

Ruta individual: `/api/categoria/:id_categoria`

Campo	Tipo SQL	Contrato de escritura
`id_categoria`	`INT`	PK; requerido en POST, inmutable en PUT
`nombre`	`NVARCHAR(80)`	Requerido en POST; modificable en PUT

Cuerpo de POST de la colección:

```
{
  "id_categoria": {{runId}},
  "nombre": "Demo categoria {{runId}}"
}
```

Cuerpo de PUT de la colección:

```
{
  "nombre": "Demo categoria {{runId}}"
}
```

A.8. marca

Marca comercial de los productos.

No depende de otras entidades. El nombre es único.

Ruta individual: `/api/marca/:id_marca`

Campo	Tipo SQL	Contrato de escritura
`id_marca`	`INT`	PK; requerido en POST, inmutable en PUT
`nombre`	`NVARCHAR(80)`	Requerido en POST; modificable en PUT

Cuerpo de POST de la colección:

```
{
  "id_marca": {{runId}},
  "nombre": "Demo marca {{runId}}"
}
```

Cuerpo de PUT de la colección:

```
{
  "nombre": "Demo marca {{runId}}"
}
```

A.9. estado_venta

Estados fijos del ciclo de venta.

1 REGISTRADA, 2 PAGADA, 3 ANULADA. Nombres y claves del sistema protegidos; escrituras de prueba devuelven 409.

Ruta individual: `/api/estado_venta/:id_estado_venta`

Campo	Tipo SQL	Contrato de escritura
`id_estado_venta`	`INT`	PK; requerido en POST, inmutable en PUT
`nombre`	`NVARCHAR(30)`	Requerido en POST; modificable en PUT

Cuerpo de POST de la colección:

```
{
  "id_estado_venta": 1,
  "nombre": "REGISTRADA"
}
```

Cuerpo de PUT de la colección:

```
{
  "nombre": "REGISTRADA"
}
```

Estos cuerpos son pruebas de rechazo: el POST duplicado y el PUT protegido deben responder 409. No crean nuevos estados.

A.10. metodo_pago

Formas de pago permitidas.

No depende de otras entidades. El nombre es único.

Ruta individual: `/api/metodo_pago/:id_metodo_pago`

Campo	Tipo SQL	Contrato de escritura
`id_metodo_pago`	`INT`	PK; requerido en POST, inmutable en PUT
`nombre`	`NVARCHAR(80)`	Requerido en POST; modificable en PUT

Cuerpo de POST de la colección:

```
{
  "id_metodo_pago": {{runId}},
  "nombre": "Demo metodo_pago {{runId}}"
}
```

Cuerpo de PUT de la colección:

```
{
  "nombre": "Demo metodo_pago {{runId}}"
}
```

A.11. persona

Datos personales compartidos por clientes y empleados.

id_municipio referencia municipio. correo es nullable y único solo cuando no es NULL.

Ruta individual: `/api/persona/:id_persona`

Campo	Tipo SQL	Contrato de escritura
`id_persona`	`INT`	PK; requerido en POST, inmutable en PUT
`nombres`	`NVARCHAR(100)`	Requerido en POST; modificable en PUT
`apellidos`	`NVARCHAR(100)`	Requerido en POST; modificable en PUT
`telefono`	`NVARCHAR(25)`	Requerido en POST; modificable en PUT
`correo`	`NVARCHAR(150)`	Campo requerido en POST; admite NULL
`direccion`	`NVARCHAR(200)`	Requerido en POST; modificable en PUT
`id_municipio`	`INT`	Requerido en POST; modificable en PUT

Cuerpo de POST de la colección:

```
{
  "id_persona": {{runId}},
  "nombres": "Cristina",
  "apellidos": "Garcia",
  "telefono": "59398441",
  "correo": null,
  "direccion": "13 Calle 3-74 zona 4",
  "id_municipio": 41
}
```

Cuerpo de PUT de la colección:

```
{
  "nombres": "Cristina"
}
```

A.12. tienda

Establecimiento donde se gestionan inventario y ventas.

Referencia municipio y tipo_tienda. Su nombre es único.

Ruta individual: `/api/tienda/:id_tienda`

Campo	Tipo SQL	Contrato de escritura
`id_tienda`	`INT`	PK; requerido en POST, inmutable en PUT
`nombre`	`NVARCHAR(120)`	Requerido en POST; modificable en PUT
`direccion`	`NVARCHAR(200)`	Requerido en POST; modificable en PUT
`telefono`	`NVARCHAR(25)`	Requerido en POST; modificable en PUT
`id_municipio`	`INT`	Requerido en POST; modificable en PUT
`id_tipo_tienda`	`INT`	Requerido en POST; modificable en PUT

Cuerpo de POST de la colección:

```
{
  "id_tienda": {{runId}},
  "nombre": "Demo tienda {{runId}}",
  "direccion": "18 Avenida 7-38 zona 15",
  "telefono": "24335942",
  "id_municipio": 41,
  "id_tipo_tienda": 1
}
```

Cuerpo de PUT de la colección:

```
{
  "nombre": "Demo tienda {{runId}}"
}
```

A.13. empleado

Asignación de una persona a tienda y cargo.

Referencia persona, tienda y cargo. Una persona tiene una sola fila de empleado; la clave alternativa empleado-tienda es referenciada por venta.

Ruta individual: `/api/empleado/:id_empleado`

Campo	Tipo SQL	Contrato de escritura
`id_empleado`	`INT`	PK; requerido en POST, inmutable en PUT
`fecha_contratacion`	`DATETIME2(0)`	Requerido en POST; modificable en PUT
`id_tienda`	`INT`	Requerido en POST; modificable en PUT
`id_cargo`	`INT`	Requerido en POST; modificable en PUT
`id_persona`	`INT`	Requerido en POST; modificable en PUT

Cuerpo de POST de la colección:

```
{
  "id_empleado": {{runId}},
  "fecha_contratacion": "2021-09-15T00:00:00",
  "id_tienda": 1,
  "id_cargo": 1,
  "id_persona": {{auxId}}
}
```

Cuerpo de PUT de la colección:

```
{
  "fecha_contratacion": "2021-09-15T00:00:00"
}
```

A.14. cliente

Identificación comercial de una persona compradora.

Referencia persona y tipo_identificacion. Persona es única y el par tipo-número de documento también.

Ruta individual: `/api/cliente/:id_cliente`

Campo	Tipo SQL	Contrato de escritura
`id_cliente`	`INT`	PK; requerido en POST, inmutable en PUT
`id_tipo_identificacion`	`INT`	Requerido en POST; modificable en PUT
`numero_identificacion`	`NVARCHAR(30)`	Requerido en POST; modificable en PUT
`id_persona`	`INT`	Requerido en POST; modificable en PUT

Cuerpo de POST de la colección:

```
{
  "id_cliente": {{runId}},
  "id_tipo_identificacion": 1,
  "numero_identificacion": "DOC{{runId}}",
  "id_persona": {{auxId}}
}
```

Cuerpo de PUT de la colección:

```
{
  "numero_identificacion": "DOC{{runId}}"
}
```

A.15. producto

Datos maestros de productos, sin stock global.

Referencia categoria y marca. El precio y las existencias se mantienen en catalogo_producto.

Ruta individual: `/api/producto/:id_producto`

Campo	Tipo SQL	Contrato de escritura
`id_producto`	`INT`	PK; requerido en POST, inmutable en PUT
`nombre`	`NVARCHAR(120)`	Requerido en POST; modificable en PUT
`descripcion`	`NVARCHAR(300)`	Requerido en POST; modificable en PUT
`id_categoria`	`INT`	Requerido en POST; modificable en PUT
`id_marca`	`INT`	Requerido en POST; modificable en PUT

Cuerpo de POST de la colección:

```
{
  "id_producto": {{runId}},
  "nombre": "Demo producto {{runId}}",
  "descripcion": "Agua pura presentacion original, categoria Bebidas",
  "id_categoria": 1,
  "id_marca": 4
}
```

Cuerpo de PUT de la colección:

```
{
  "nombre": "Demo producto {{runId}}"
}
```

A.16. catalogo_producto

Precio vigente y stock de un producto en una tienda.

Referencia tienda y producto. Precio positivo, stock no negativo y clave compuesta. No se elimina si hay detalles asociados.

Ruta individual: `/api/catalogo_producto/:id_tienda/:id_producto`

Campo	Tipo SQL	Contrato de escritura
precio_vigente	DECIMAL(12,2)	Requerido en POST; modificable en PUT
existencia_actual	INT	Requerido en POST; modificable en PUT
id_tienda	INT	PK; requerido en POST, inmutable en PUT
id_producto	INT	PK; requerido en POST, inmutable en PUT

Cuerpo de POST de la colección:

```
{
  "id_tienda": 1,
  "id_producto": {{auxId}},
  "precio_vigente": 25,
  "existencia_actual": 10
}
```

Cuerpo de PUT de la colección:

```
{
  "precio_vigente": 25
}
```

A.17. venta

Encabezado de la transacción comercial.

Referencia cliente, estado_venta y el par empleado-tienda. Debe comenzar REGISTRADA; consultar devuelve estado y totales calculados.

Ruta individual: `/api/venta/:id_venta`

Campo	Tipo SQL	Contrato de escritura
`id_venta`	`INT`	PK; requerido en POST, inmutable en PUT
`fecha_venta`	`DATETIME2(0)`	Requerido en POST; modificable en PUT
`id_tienda`	`INT`	Requerido en POST; modificable en PUT
`id_empleado`	`INT`	Requerido en POST; modificable en PUT
`id_cliente`	`INT`	Requerido en POST; modificable en PUT
`id_estado_venta`	`INT`	Requerido en POST; modificable en PUT

Cuerpo de POST de la colección:

```
{
  "id_venta": {{runId}},
  "fecha_venta": "2026-10-16T12:00:00",
  "id_tienda": 1,
  "id_empleado": 1,
  "id_cliente": 1,
  "id_estado_venta": 1
}
```

Cuerpo de PUT de la colección:

```
{
  "fecha_venta": "2026-10-16T12:00:00"
}
```

A.18. detalle_venta

Producto, cantidad y precio histórico de una venta.

Referencia venta y producto. El trigger exige catálogo de la tienda, stock suficiente y venta registrada sin pagos. subtotal=cantidad*precio_unitario.

Ruta individual: `/api/detalle_venta/:id_venta/:id_producto`

Campo	Tipo SQL	Contrato de escritura
`cantidad`	`INT`	Requerido en POST; modificable en PUT
`precio_unitario`	`DECIMAL(12,2)`	Requerido en POST; modificable en PUT
`subtotal`	`DECIMAL(12,2)`	Requerido en POST; modificable en PUT
`id_venta`	`INT`	PK; requerido en POST, inmutable en PUT
`id_producto`	`INT`	PK; requerido en POST, inmutable en PUT

Cuerpo de POST de la colección:

```
{
  "id_venta": {{auxId}},
  "id_producto": {{auxId}},
  "cantidad": 2,
  "precio_unitario": 25,
  "subtotal": 50
}
```

Cuerpo de PUT de la colección:

```
{
  "cantidad": 3,
  "subtotal": 75
}
```

La subcarpeta incluye una petición previa de preparación. Ejecutarla antes del CRUD y conservar el orden de toda la colección.

A.19. pago

Abono aplicado a una venta por un método de pago.

Referencia venta y metodo_pago. Monto positivo. No se permite trasladar id_venta en PUT; su trigger recalcula el estado.

Ruta individual: `/api/pago/:id_pago`

Campo	Tipo SQL	Contrato de escritura
`id_pago`	`INT`	PK; requerido en POST, inmutable en PUT
`monto`	`DECIMAL(12,2)`	Requerido en POST; modificable en PUT
`id_metodo_pago`	`INT`	Requerido en POST; modificable en PUT
`id_venta`	`INT`	Requerido en POST; inmutable en PUT

Cuerpo de POST de la colección:

```
{
  "id_pago": {{runId}},
  "id_venta": {{auxId}},
  "id_metodo_pago": 1,
  "monto": 20
}
```

Cuerpo de PUT de la colección:

```
{
  "monto": 50
}
```

La subcarpeta incluye una petición previa de preparación. Ejecutarla antes del CRUD y conservar el orden de toda la colección.

B. Matriz de todas las peticiones de Postman

Esta matriz se genera desde los scripts de prueba de la colección. Los códigos de rechazo son resultados esperados. Las verificaciones adicionales indican qué valor funcional comprobar junto con el estado HTTP. Una ejecución completa utiliza 144 peticiones y 155 aserciones en la versión documentada.

B.1. 00 Preparacion

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Iniciar ejecucion y comprobar conexion	GET	200	/health
Verificar carga historica detalle.filas=9068	GET	200	/admin/importacion
Persona auxiliar	POST	201	/persona
Producto auxiliar	POST	201	/producto
Venta auxiliar	POST	201	/venta

B.2. 01 CRUD por entidad

B.2.1. pais

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Crear	POST	201	/pais
Listar	GET	200	/pais?limit=5&offset=0
Buscar por clave	GET	200	/pais/{{runId}}
Actualizar	PUT	200	/pais/{{runId}}
Eliminar	DELETE	204	/pais/{{runId}}
Confirmar eliminacion	GET	404	/pais/{{runId}}

B.2.2. departamento

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Crear	POST	201	/departamento
Listar	GET	200	/departamento?limit=5&offset=0
Buscar por clave	GET	200	/departamento/{{runId}}
Actualizar	PUT	200	/departamento/{{runId}}
Eliminar	DELETE	204	/departamento/{{runId}}
Confirmar eliminacion	GET	404	/departamento/{{runId}}

B.2.3. municipio

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Crear	POST	201	/municipio
Listar	GET	200	/municipio?limit=5&offset=0
Buscar por clave	GET	200	/municipio/{{runId}}
Actualizar	PUT	200	/municipio/{{runId}}
Eliminar	DELETE	204	/municipio/{{runId}}
Confirmar eliminacion	GET	404	/municipio/{{runId}}

B.2.4. tipo_tienda

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Crear	POST	201	/tipo_tienda
Listar	GET	200	/tipo_tienda?limit=5&offset=0
Buscar por clave	GET	200	/tipo_tienda/{{runId}}
Actualizar	PUT	200	/tipo_tienda/{{runId}}
Eliminar	DELETE	204	/tipo_tienda/{{runId}}
Confirmar eliminacion	GET	404	/tipo_tienda/{{runId}}

B.2.5. tipo_identificacion

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Crear	POST	201	/tipo_identificacion
Listar	GET	200	/tipo_identificacion?limit=5&offset=0
Buscar por clave	GET	200	/tipo_identificacion/{{runId}}
Actualizar	PUT	200	/tipo_identificacion/{{runId}}
Eliminar	DELETE	204	/tipo_identificacion/{{runId}}
Confirmar eliminacion	GET	404	/tipo_identificacion/{{runId}}

B.2.6. cargo

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Crear	POST	201	/cargo
Listar	GET	200	/cargo?limit=5&offset=0
Buscar por clave	GET	200	/cargo/{{runId}}
Actualizar	PUT	200	/cargo/{{runId}}
Eliminar	DELETE	204	/cargo/{{runId}}
Confirmar eliminacion	GET	404	/cargo/{{runId}}

B.2.7. categoria

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Crear	POST	201	/categoria
Listar	GET	200	/categoria?limit=5&offset=0
Buscar por clave	GET	200	/categoria/{{runId}}
Actualizar	PUT	200	/categoria/{{runId}}
Eliminar	DELETE	204	/categoria/{{runId}}
Confirmar eliminacion	GET	404	/categoria/{{runId}}

B.2.8. marca

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Crear	POST	201	/marca
Listar	GET	200	/marca?limit=5&offset=0
Buscar por clave	GET	200	/marca/{{runId}}
Actualizar	PUT	200	/marca/{{runId}}
Eliminar	DELETE	204	/marca/{{runId}}
Confirmar eliminacion	GET	404	/marca/{{runId}}

B.2.9. estado_venta (catalogo protegido)

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Listar	GET	200	/estado_venta
Buscar	GET	200	/estado_venta/1
POST duplicado	POST	409	/estado_venta
PUT protegido	PUT	409	/estado_venta/1
DELETE protegido	DELETE	409	/estado_venta/1

B.2.10. metodo_pago

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Crear	POST	201	/metodo_pago
Listar	GET	200	/metodo_pago?limit=5&offset=0
Buscar por clave	GET	200	/metodo_pago/{{runId}}
Actualizar	PUT	200	/metodo_pago/{{runId}}
Eliminar	DELETE	204	/metodo_pago/{{runId}}
Confirmar eliminacion	GET	404	/metodo_pago/{{runId}}

B.2.11. persona

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Crear	POST	201	/persona
Listar	GET	200	/persona?limit=5&offset=0
Buscar por clave	GET	200	/persona/{{runId}}
Actualizar	PUT	200	/persona/{{runId}}
Eliminar	DELETE	204	/persona/{{runId}}
Confirmar eliminacion	GET	404	/persona/{{runId}}

B.2.12. tienda

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Crear	POST	201	/tienda
Listar	GET	200	/tienda?limit=5&offset=0
Buscar por clave	GET	200	/tienda/{{runId}}
Actualizar	PUT	200	/tienda/{{runId}}
Eliminar	DELETE	204	/tienda/{{runId}}
Confirmar eliminacion	GET	404	/tienda/{{runId}}

B.2.13. empleado

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Crear	POST	201	/empleado
Listar	GET	200	/empleado?limit=5&offset=0
Buscar por clave	GET	200	/empleado/{{runId}}
Actualizar	PUT	200	/empleado/{{runId}}
Eliminar	DELETE	204	/empleado/{{runId}}
Confirmar eliminacion	GET	404	/empleado/{{runId}}

B.2.14. cliente

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Crear	POST	201	/cliente
Listar	GET	200	/cliente?limit=5&offset=0
Buscar por clave	GET	200	/cliente/{{runId}}
Actualizar	PUT	200	/cliente/{{runId}}
Eliminar	DELETE	204	/cliente/{{runId}}
Confirmar eliminacion	GET	404	/cliente/{{runId}}

B.2.15. producto

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Crear	POST	201	/producto
Listar	GET	200	/producto?limit=5&offset=0
Buscar por clave	GET	200	/producto/{{runId}}
Actualizar	PUT	200	/producto/{{runId}}
Eliminar	DELETE	204	/producto/{{runId}}
Confirmar eliminacion	GET	404	/producto/{{runId}}

B.2.16. catalogo_producto

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Crear	POST	201	/catalogo_producto
Listar	GET	200	/catalogo_producto?limit=5&offset=0
Buscar por clave	GET	200	/catalogo_producto/1/{{auxId}}
Actualizar	PUT	200	/catalogo_producto/1/{{auxId}}
Eliminar	DELETE	204	/catalogo_producto/1/{{auxId}}
Confirmar eliminacion	GET	404	/catalogo_producto/1/{{auxId}}

B.2.17. venta

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Crear	POST	201	/venta
Listar	GET	200	/venta?limit=5&offset=0
Buscar por clave	GET	200	/venta/{{runId}}
Actualizar	PUT	200	/venta/{{runId}}
Eliminar	DELETE	204	/venta/{{runId}}
Confirmar eliminacion	GET	404	/venta/{{runId}}

B.2.18. detalle_venta

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Preparar inventario auxiliar	POST	201	/catalogo_producto
Crear	POST	201	/detalle_venta
Listar	GET	200	/detalle_venta?limit=5&offset=0
Buscar por clave	GET	200	/detalle_venta/{{auxId}}/{{auxId}}
Actualizar	PUT	200	/detalle_venta/{{auxId}}/{{auxId}}
Eliminar	DELETE	204	/detalle_venta/{{auxId}}/{{auxId}}
Confirmar eliminacion	GET	404	/detalle_venta/{{auxId}}/{{auxId}}

B.2.19. pago

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Preparar detalle para abonos	POST	201	/detalle_venta
Crear	POST	201	/pago
Listar	GET	200	/pago?limit=5&offset=0
Buscar por clave	GET	200	/pago/{{runId}}
Actualizar	PUT	200	/pago/{{runId}}
Eliminar	DELETE	204	/pago/{{runId}}
Confirmar eliminacion	GET	404	/pago/{{runId}}

B.3. 02 Limpieza auxiliares

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Eliminar detalle auxiliar	DELETE	204	/detalle_venta/{{auxId}}/{{auxId}}
Eliminar venta auxiliar	DELETE	204	/venta/{{auxId}}
Eliminar catalogo auxiliar	DELETE	204	/catalogo_producto/1/{{auxId}}
Eliminar producto auxiliar	DELETE	204	/producto/{{auxId}}
Eliminar persona auxiliar	DELETE	204	/persona/{{auxId}}

B.4. 03 Triggers

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Producto demostracion	POST	201	/producto
Inventario inicial 10	POST	201	/catalogo_producto
Venta registrada	POST	201	/venta
Stock insuficiente: pedir 11 error='Stock insuficiente'	POST	400	/detalle_venta
Inventario sigue en 10 existencia_actual=10	GET	200	/catalogo_producto/1/{{runId}}
Venta valida: pedir 2	POST	201	/detalle_venta
Inventario reducido a 8 existencia_actual=8	GET	200	/catalogo_producto/1/{{runId}}
Primer abono 20	POST	201	/pago
Estado aun REGISTRADA estado='REGISTRADA'	GET	200	/venta/{{runId}}
Segundo abono 30	POST	201	/pago
Estado automatico PAGADA estado='PAGADA'; saldo_pendiente=0	GET	200	/venta/{{runId}}
Otra venta para anular	POST	201	/venta
Detalle para anular	POST	201	/detalle_venta
Anular sin pagos	PUT	200	/venta/{{auxId}}
Inventario devuelto una vez existencia_actual=8	GET	200	/catalogo_producto/1/{{runId}}
Repetir anulacion rechazada	PUT	409	/venta/{{auxId}}

B.5. 04 Vistas

Petición / validación	Método	HTTP	Ruta relativa a baseUrl
Vista ventas-ubicacion data es un arreglo	GET	200	/reportes/ventas-ubicacion
Vista top-productos data es un arreglo	GET	200	/reportes/top-productos
Vista saldos-pendientes data es un arreglo	GET	200	/reportes/saldos-pendientes

C. Implementación SQL de las vistas y triggers

Los listados siguientes se incluyen directamente desde init.sql para mantener correspondencia con el código entregado. El esquema completo de tablas, logins y permisos se consulta en ese archivo. Estos fragmentos explican el comportamiento; no se ejecutan manualmente sobre los datos durante la calificación.

C.1. Vista_Ventas_Ubicacion

```
CREATE OR ALTER VIEW dbo.Vista_Ventas_Ubicacion AS
WITH totales AS (
    SELECT id_venta, SUM(subtotal) AS total FROM dbo.detalle_venta GROUP BY id_venta
)
SELECT t.id_tienda, t.nombre AS tienda, m.nombre AS municipio,
       d.nombre AS departamento, p.nombre AS pais,
       COUNT_BIG(v.id_venta) AS cantidad_ventas,
       COALESCE(SUM(tot.total), 0) AS total_facturado
FROM dbo.venta v
JOIN dbo.estado_venta ev ON ev.id_estado_venta = v.id_estado_venta
JOIN dbo.tienda t ON t.id_tienda = v.id_tienda
JOIN dbo.municipio m ON m.id_municipio = t.id_municipio
JOIN dbo.departamento d ON d.id_departamento = m.id_departamento
JOIN dbo.pais p ON p.id_pais = d.id_pais
LEFT JOIN totales tot ON tot.id_venta = v.id_venta
WHERE ev.nombre <> N'ANULADA'
GROUP BY t.id_tienda, t.nombre, m.nombre, d.nombre, p.nombre;
```

C.2. Vista_Top_Productos

```
CREATE OR ALTER VIEW dbo.Vista_Top_Productos AS
SELECT p.id_producto AS codigo_producto, p.nombre, c.nombre AS categoria,
       m.nombre AS marca, SUM(CONVERT(BIGINT,d.cantidad)) AS unidades_vendidas,
       SUM(d.subtotal) AS monto_total
FROM dbo.detalle_venta d
JOIN dbo.venta v ON v.id_venta = d.id_venta
JOIN dbo.estado_venta ev ON ev.id_estado_venta = v.id_estado_venta
JOIN dbo.producto p ON p.id_producto = d.id_producto
JOIN dbo.categoria c ON c.id_categoria = p.id_categoria
JOIN dbo.marca m ON m.id_marca = p.id_marca
WHERE ev.nombre <> N'ANULADA'
GROUP BY p.id_producto, p.nombre, c.nombre, m.nombre;
```

C.3. Vista_Consulta_Ventas

```
CREATE OR ALTER VIEW dbo.Vista_Consulta_Ventas AS
WITH totales AS (
    SELECT id_venta, SUM(subtotal) AS total FROM dbo.detalle_venta GROUP BY id_venta
), pagos AS (
    SELECT id_venta, SUM(monto) AS pagado FROM dbo.pago GROUP BY id_venta
)
SELECT v.*, e.nombre AS estado, COALESCE(t.total,0) AS total_venta,
       COALESCE(p.pagado,0) AS total_pagado,
       COALESCE(t.total,0)-COALESCE(p.pagado,0) AS saldo_pendiente
FROM dbo.venta v
JOIN dbo.estado_venta e ON e.id_estado_venta = v.id_estado_venta
LEFT JOIN totales t ON t.id_venta = v.id_venta
LEFT JOIN pagos p ON p.id_venta = v.id_venta;
```

C.4. Vista_Saldos_Pendientes

```
CREATE OR ALTER VIEW dbo.Vista_Saldos_Pendientes AS
SELECT id_venta, total_venta, total_pagado, saldo_pendiente
FROM dbo.Vista_Consulta_Ventas WHERE estado = N'REGISTRADA';
```

C.5. Insert_Detalle

```
CREATE OR ALTER TRIGGER dbo.Insert_Detalle ON dbo.detalle_venta
AFTER INSERT, UPDATE, DELETE AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    IF IS_ROLEMEMBER(N'importador') = 1
       AND TRY_CONVERT(INT, SESSION_CONTEXT(N'importacion_historica')) = 1 RETURN;

    DECLARE @ventas TABLE (id INT PRIMARY KEY);
    INSERT @ventas SELECT id_venta FROM inserted UNION SELECT id_venta FROM deleted;
    DECLARE @lock INT;
    SELECT @lock = v.id_venta FROM dbo.venta v WITH (UPDLOCK, HOLDLOCK)
    JOIN @ventas a ON a.id = v.id_venta ORDER BY v.id_venta;
    IF EXISTS (
        SELECT 1 FROM @ventas a JOIN dbo.venta v ON v.id_venta=a.id
        JOIN dbo.estado_venta e ON e.id_estado_venta=v.id_estado_venta
        WHERE e.nombre <> N'REGISTRADA' OR EXISTS(SELECT 1 FROM dbo.pago p WHERE p.id_venta=a.id)
    ) THROW 51002, N'Solo se permiten detalles en ventas REGISTRADA sin pagos', 1;

    DECLARE @delta TABLE (id_tienda INT, id_producto INT, cantidad BIGINT, PRIMARY KEY(id_tienda,id_producto));
    INSERT @delta
    SELECT v.id_tienda, x.id_producto, SUM(x.cantidad)
    FROM (
        SELECT id_venta,id_producto,CONVERT(BIGINT,cantidad) cantidad FROM inserted
        UNION ALL
        SELECT id_venta,id_producto,-CONVERT(BIGINT,cantidad) FROM deleted
    ) x JOIN dbo.venta v ON v.id_venta=x.id_venta
    GROUP BY v.id_tienda,x.id_producto;

    UPDATE c WITH (UPDLOCK, HOLDLOCK)
       SET existencia_actual = c.existencia_actual - d.cantidad
    FROM dbo.catalogo_producto c JOIN @delta d
      ON d.id_tienda=c.id_tienda AND d.id_producto=c.id_producto
    WHERE CONVERT(BIGINT,c.existencia_actual) >= d.cantidad;
    IF @@ROWCOUNT <> (SELECT COUNT(*) FROM @delta)
        THROW 51001, N'Stock insuficiente', 1;
END;
```

C.6. Insert_Pago

```
CREATE OR ALTER TRIGGER dbo.Insert_Pago ON dbo.pago
AFTER INSERT, UPDATE, DELETE AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    IF IS_ROLEMEMBER(N'importador') = 1
       AND TRY_CONVERT(INT, SESSION_CONTEXT(N'importacion_historica')) = 1 RETURN;
    DECLARE @ventas TABLE (id INT PRIMARY KEY);
    INSERT @ventas SELECT id_venta FROM inserted UNION SELECT id_venta FROM deleted;
    DECLARE @lock INT;
    SELECT @lock=v.id_venta FROM dbo.venta v WITH(UPDLOCK,HOLDLOCK)
    JOIN @ventas a ON a.id=v.id_venta ORDER BY v.id_venta;
    IF EXISTS (
        SELECT 1 FROM @ventas a JOIN dbo.venta v ON v.id_venta=a.id
        JOIN dbo.estado_venta e ON e.id_estado_venta=v.id_estado_venta
        WHERE e.nombre=N'ANULADA'
    ) THROW 51002, N'No se permiten pagos en ventas anuladas', 1;
    IF EXISTS (
        SELECT 1 FROM inserted i WHERE NOT EXISTS (
            SELECT 1 FROM dbo.detalle_venta d WHERE d.id_venta=i.id_venta
        )
    ) THROW 51002, N'La venta debe tener detalles antes de recibir pagos', 1;

    UPDATE v SET id_estado_venta = e.id_estado_venta
    FROM dbo.venta v JOIN @ventas a ON a.id=v.id_venta
    CROSS APPLY (SELECT COALESCE(SUM(subtotal),0) total FROM dbo.detalle_venta WHERE id_venta=v.id_venta) d
    CROSS APPLY (SELECT COALESCE(SUM(monto),0) total FROM dbo.pago WHERE id_venta=v.id_venta) p
    JOIN dbo.estado_venta e ON e.nombre = CASE WHEN d.total > 0 AND p.total >= d.total THEN N'PAGADA' ELSE N'REGISTRADA'
     END;
END;
```

C.7. Validar_Venta

```
CREATE OR ALTER TRIGGER dbo.Validar_Venta ON dbo.venta
AFTER INSERT, UPDATE, DELETE AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    IF IS_ROLEMEMBER(N'importador') = 1
       AND TRY_CONVERT(INT, SESSION_CONTEXT(N'importacion_historica')) = 1 RETURN;
    IF TRIGGER_NESTLEVEL(OBJECT_ID(N'dbo.Insert_Pago')) > 0 RETURN;
    IF EXISTS (
        SELECT 1 FROM inserted i LEFT JOIN deleted d ON d.id_venta=i.id_venta
        JOIN dbo.estado_venta e ON e.id_estado_venta=i.id_estado_venta
        WHERE d.id_venta IS NULL AND e.nombre <> N'REGISTRADA'
    ) THROW 51002,N'Una venta nueva debe iniciar REGISTRADA',1;
    IF EXISTS (
        SELECT 1 FROM inserted i JOIN deleted d ON d.id_venta=i.id_venta
        JOIN dbo.estado_venta anterior ON anterior.id_estado_venta=d.id_estado_venta
        JOIN dbo.estado_venta nuevo ON nuevo.id_estado_venta=i.id_estado_venta
        WHERE (anterior.nombre=N'ANULADA')
           OR (i.id_estado_venta<>d.id_estado_venta AND NOT (
                anterior.nombre=N'REGISTRADA' AND nuevo.nombre=N'ANULADA'
                AND NOT EXISTS(SELECT 1 FROM dbo.pago p WHERE p.id_venta=i.id_venta)))
           OR ((i.id_tienda<>d.id_tienda OR i.id_empleado<>d.id_empleado OR i.id_cliente<>d.id_cliente OR i.fecha_venta<>d.
     fecha_venta)
               AND (EXISTS(SELECT 1 FROM dbo.detalle_venta dv WHERE dv.id_venta=i.id_venta)
                 OR EXISTS(SELECT 1 FROM dbo.pago p WHERE p.id_venta=i.id_venta)))
    ) THROW 51002,N'Cambio de venta no permitido: estado automatico, venta anulada o registros asociados',1;

    -- Devolver stock solo en la transicion REGISTRADA -> ANULADA.
    ;WITH devolucion AS (
        SELECT i.id_tienda,dv.id_producto,SUM(CONVERT(BIGINT,dv.cantidad)) cantidad
        FROM inserted i JOIN deleted d ON d.id_venta=i.id_venta
        JOIN dbo.estado_venta e ON e.id_estado_venta=i.id_estado_venta
        JOIN dbo.detalle_venta dv ON dv.id_venta=i.id_venta
        WHERE i.id_estado_venta<>d.id_estado_venta AND e.nombre=N'ANULADA'
        GROUP BY i.id_tienda,dv.id_producto
    )
    UPDATE c SET existencia_actual=c.existencia_actual+x.cantidad
    FROM dbo.catalogo_producto c JOIN devolucion x ON x.id_tienda=c.id_tienda AND x.id_producto=c.id_producto;
END;
```

C.8. Proteger_Catalogo

```
CREATE OR ALTER TRIGGER dbo.Proteger_Catalogo ON dbo.catalogo_producto
AFTER UPDATE, DELETE AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1 FROM deleted d
        WHERE NOT EXISTS (SELECT 1 FROM inserted i WHERE i.id_tienda=d.id_tienda AND i.id_producto=d.id_producto)
          AND EXISTS (SELECT 1 FROM dbo.detalle_venta dv JOIN dbo.venta v ON v.id_venta=dv.id_venta
                      WHERE v.id_tienda=d.id_tienda AND dv.id_producto=d.id_producto)
    ) THROW 51002,N'El catalogo tiene detalles de venta asociados',1;
END;
```

C.9. Proteger_Estados

```
CREATE OR ALTER TRIGGER dbo.Proteger_Estados ON dbo.estado_venta
AFTER INSERT, UPDATE, DELETE AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS(SELECT 1 FROM deleted) THROW 51002,N'Los estados del sistema no se modifican ni eliminan',1;
    IF EXISTS(SELECT 1 FROM inserted WHERE (id_estado_venta=1 AND nombre<>N'REGISTRADA')
       OR (id_estado_venta=2 AND nombre<>N'PAGADA') OR (id_estado_venta=3 AND nombre<>N'ANULADA')
       OR id_estado_venta NOT IN(1,2,3)) THROW 51002,N'Estado del sistema invalido',1;
END;
```

D. Fuentes y trazabilidad

1. Enunciado oficial: ProyectoFase2_SBD1_2S2026.pdf, sección 4 de entregables y rúbrica de documentación. Se incluye en docs.
2. Hoja de calificación: Hoja_Calificacion_Proyecto_Fase_2_BD1.pdf. Se incluye en docs; documentación vale 15 puntos.
3. Modelo y dataset de la fase 1: documentos previos del repositorio y origen registrado en data/manifest.json.
4. Código fuente de la fase 2: init.sql; docker-compose.yml; src/config; src/entities; src/controllers; src/routes; src/-services; scripts y tests.
5. Colección entregada: postman/ComercialLaEstrella.postman_collection.json y local.postman_environment.json. De aquí se obtienen los ejemplos y códigos de la matriz.
6. Evidencia original: docs/evidencias/postman-oficial.json, integracion.json, persistencia.json y validacion.json. Las imágenes de inventario y pagos representan respuestas de ese registro.
7. Repositorio del proyecto: https://github.com/ricardious/SBD1_2S2026. La implementación se integró a develop y main mediante los PR 1 y 2; el presente manual amplía la documentación de esa implementación.